Campus Maintenance System - Full Project Documentation

Generated on: **2026-05-02 14:12:47**

Scope

* This document covers all detected project code files (`.java`, `.js`, `.jsx`, `.ts`, `.tsx`, `.cpp`, `.c`, `.h`, `.hpp`, `.sql`).
* For each file, the document lists purpose and detected functions/methods/classes.

Project Modules

* `backend/`: Spring Boot API, business logic, repositories, entities, security, schedulers.
* `frontend/`: React + Vite web app, dashboards, hooks, services, utility modules.
* `database/`: SQL schema and seed scripts.
* `backend/src/main/java/com/smartcampus/maintenance/optimization/`: Java optimization routines used for scoring and safe image handling.
* `tests/`: Integration and E2E flows.

Code Inventory

* `java`: 207 file(s)
* `js`: 36 file(s)
* `jsx`: 62 file(s)
* `sql`: 8 file(s)
* `ts`: 1 file(s)
* `tsx`: 9 file(s)

File-by-File Function Documentation

`backend/src/main/java/com/smartcampus/maintenance/SmartCampusMaintenanceApplication.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance`
* Types and Methods:
  - `class SmartCampusMaintenanceApplication`
    - Methods:
      - `public static void main(String[] args)`

`backend/src/main/java/com/smartcampus/maintenance/config/DataSeeder.java`

* Language: `java`
* Purpose: Application/configuration code
* Package: `com.smartcampus.maintenance.config`
* Types and Methods:
  - `class DataSeeder`
    - Methods:
      - `CommandLineRunner seedData(UserRepository userRepository, BuildingRepository buildingRepository, RequestTypeRepository requestTypeRepository, TicketRepository ticketRepository, TicketLogRepository ticketLogRepository, TicketRatingRepository ticketRatingRepository, NotificationRepository notificationRepository, PasswordEncoder passwordEncoder, boolean bootstrapAdmin, boolean seedDemoData, String adminUsername, String adminEmail, String adminFullName, String adminPassword, boolean syncExistingAdmin, String demoPassword)`
      - `private Building ensureBuilding(BuildingRepository buildingRepository, String name, String code, int floors, int sortOrder)`
      - `private RequestType requireRequestType(RequestTypeRepository requestTypeRepository, TicketCategory category)`
      - `private User createUser(UserRepository userRepository, PasswordEncoder encoder, String username, String

email,
String fullName, Role role, String rawPassword)`
- `private User ensureAdminUser(UserRepository userRepository, PasswordEncoder encoder, String adminUsername,
String adminEmail, String adminFullName, String adminPassword, boolean syncExistingAdmin)`
- `private void addLog(TicketLogRepository ticketLogRepository, Ticket ticket, TicketStatus oldStatus, TicketStatus newStatus, User actor, String note)`
- `private void addRating(TicketRatingRepository ticketRatingRepository, Ticket ticket, User ratedBy, int stars, String comment)`
- `private void seedNotification(NotificationRepository notificationRepository, User user, String title, String message, NotificationType type, String linkUrl)`
- `private void transition(TicketRepository ticketRepository, TicketLogRepository ticketLogRepository, Ticket ticket, TicketStatus target, User actor, String note)`
- `protected Ticket seedTicket(TicketRepository ticketRepository, TicketLogRepository ticketLogRepository, User createdBy, User assignedTo, User adminActor, User maintenanceActor, String title, String description, RequestType requestType, Building building, String location, UrgencyLevel urgency, LocalDateTime createdAt, TicketStatus finalStatus)`

`backend/src/main/java/com/smartcampus/maintenance/config/EmailConfigurationValidator.java`

* Language: `java`
* Purpose: Application/configuration code
* Package: `com.smartcampus.maintenance.config`
* Types and Methods:
  - `class EmailConfigurationValidator`
    - Constructors:
      - `public EmailConfigurationValidator(boolean emailEnabled, String fromAddress, String mailHost, int mailPort, String mailUsername, String mailPassword)`
    - Methods:
      - `private boolean isGmailHost(String host)`
      - `public void validate()`

`backend/src/main/java/com/smartcampus/maintenance/config/ProductionDeploymentValidator.java`

* Language: `java`
* Purpose: Application/configuration code
* Package: `com.smartcampus.maintenance.config`
* Types and Methods:
  - `class ProductionDeploymentValidator`
    - Constructors:
      - `public ProductionDeploymentValidator(String jwtSecret, boolean h2ConsoleEnabled, boolean bootstrapAdmin, boolean seedDemoData, boolean syncExistingAdmin, String adminPassword, String datasourceUrl, String frontendBaseUrl,
List<String> corsAllowedOrigins, boolean emailEnabled, String mailHost, String mailUsername, String mailPassword, boolean captchaEnabled, String captchaSecretKey, boolean rateLimitRedisEnabled, boolean refreshCookieSecure)`
    - Methods:
      - `private boolean isLocalhostOrigin(String origin)`
      - `private boolean isWeakBootstrapAdminPassword(String password)`
      - `public void run(ApplicationArguments args)`
      - `void validate()`

`backend/src/main/java/com/smartcampus/maintenance/config/RedisRateLimitConfiguration.java`

* Language: `java`
* Purpose: Application/configuration code
* Package: `com.smartcampus.maintenance.config`
* Types and Methods:
  - `class RedisRateLimitConfiguration`
    - Methods:

    - `public LettuceConnectionFactory redisConnectionFactory(String host, int port, String password)`
    - `public StringRedisTemplate stringRedisTemplate(LettuceConnectionFactory redisConnectionFactory)`

`backend/src/main/java/com/smartcampus/maintenance/config/SecurityConfig.java`

* Language: `java`
* Purpose: Application/configuration code
* Package: `com.smartcampus.maintenance.config`
* Types and Methods:
  - `class SecurityConfig`
    - Constructors:
      - `public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter, CustomUserDetailsService userDetailsService, ObjectMapper objectMapper, List<String> allowedOrigins, boolean h2ConsoleEnabled)`
    - Methods:
      - `public AuthenticationManager authenticationManager(AuthenticationConfiguration authenticationConfiguration)`
      - `public AuthenticationProvider authenticationProvider()`
      - `public CorsConfigurationSource corsConfigurationSource()`
      - `public PasswordEncoder passwordEncoder()`
      - `public SecurityFilterChain securityFilterChain(HttpSecurity http)`

`backend/src/main/java/com/smartcampus/maintenance/controller/AdminConfigurationController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class AdminConfigurationController`
    - Constructors:
      - `public AdminConfigurationController(CurrentUserService currentUserService, BuildingService buildingService, CatalogService catalogService)`
    - Methods:
      - `public BuildingResponse createBuilding(BuildingCreateRequest request)`
      - `public BuildingResponse updateBuilding(Long id, BuildingUpdateRequest request)`
      - `public List<BuildingResponse> getBuildings()`
      - `public List<RequestTypeResponse> getRequestTypes()`
      - `public List<SupportCategoryResponse> getSupportCategories()`
      - `public RequestTypeResponse createRequestType(RequestTypeCreateRequest request)`
      - `public RequestTypeResponse updateRequestType(Long id, RequestTypeUpdateRequest request)`
      - `public SupportCategoryResponse createSupportCategory(SupportCategoryCreateRequest request)`
      - `public SupportCategoryResponse updateSupportCategory(Long id, SupportCategoryUpdateRequest request)`

`backend/src/main/java/com/smartcampus/maintenance/controller/AnalyticsController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class AnalyticsController`
    - Constructors:
      - `public AnalyticsController(AnalyticsService analyticsService, PublicLandingConfigService publicLandingConfigService, SlaService slaService, ReportService reportService, CurrentUserService currentUserService)`
    - Methods:
      - `public AnalyticsSummaryResponse getSummary()`
      - `public List<CrewPerformanceResponse> getCrewPerformance()`
      - `public List<TopBuildingResponse> getTopBuildings()`
      - `public PublicLandingConfigResponse getPublicConfig()`
      - `public PublicLandingStatsResponse getPublicSummary()`

- `public ResolutionTimeResponse getResolutionTime()`
- `public ResponseEntity<byte[]> exportCsv()`
- `public SlaComplianceResponse getSlaCompliance()`

`backend/src/main/java/com/smartcampus/maintenance/controller/AnnouncementController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class AnnouncementController`
    - Constructors:
      - `public AnnouncementController(AnnouncementService announcementService, CurrentUserService currentUserService)`
    - Methods:
      - `public AnnouncementResponse create(AnnouncementCreateRequest request)`
      - `public List<AnnouncementResponse> getAllAnnouncements()`
      - `public List<AnnouncementResponse> getAnnouncements()`
      - `public void toggleActive(Long id)`

`backend/src/main/java/com/smartcampus/maintenance/controller/AuthController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class AuthController`
    - Constructors:
      - `public AuthController(AuthService authService, CurrentUserService currentUserService, RequestMetadataResolver requestMetadataResolver, PublicEndpointSecurityService publicEndpointSecurityService, RefreshCookieService refreshCookieService)`
    - Methods:
      - `private String readRefreshToken(HttpServletRequest request)`
      - `public CurrentUserResponse me()`
      - `public Map<String, String> acceptStaffInvite(AcceptStaffInviteRequest request, HttpServletRequest servletRequest)`
      - `public Map<String, String> forgotPassword(ForgotPasswordRequest request, HttpServletRequest servletRequest)`
      - `public Map<String, String> register(RegisterRequest request, HttpServletRequest servletRequest)`
      - `public Map<String, String> resendMfa(ResendMfaRequest request, HttpServletRequest servletRequest)`
      - `public Map<String, String> resendVerification(ResendVerificationRequest request, HttpServletRequest servletRequest)`
      - `public Map<String, String> resetPassword(ResetPasswordRequest request, HttpServletRequest servletRequest)`
      - `public Map<String, String> verifyEmail(VerifyEmailRequest request, HttpServletRequest servletRequest)`
      - `public ResponseEntity<AuthResponse> login(LoginRequest request, HttpServletRequest servletRequest)`
      - `public ResponseEntity<AuthResponse> refresh(HttpServletRequest servletRequest)`
      - `public ResponseEntity<AuthResponse> verifyMfa(VerifyMfaRequest request, HttpServletRequest servletRequest)`
      - `public ResponseEntity<Map<String, String>> logout(HttpServletRequest servletRequest)`
      - `public UsernameSuggestionsResponse getUsernameSuggestions(String username, String fullName)`

`backend/src/main/java/com/smartcampus/maintenance/controller/BuildingController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class BuildingController`

  - Constructors:
    - `public BuildingController(BuildingService buildingService, CurrentUserService currentUserService)`
  - Methods:
    - `public BuildingResponse createBuilding(BuildingCreateRequest request)`
    - `public BuildingResponse updateBuilding(Long id, BuildingUpdateRequest request)`
    - `public List<BuildingResponse> getBuildings(boolean includeArchived)`

`backend/src/main/java/com/smartcampus/maintenance/controller/CatalogController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class CatalogController`
    - Constructors:
      - `public CatalogController(CatalogService catalogService, CatalogEventStreamService catalogEventStreamService)`
    - Methods:
      - `public List<RequestTypeResponse> getRequestTypes(String serviceDomainKey)`
      - `public List<ServiceDomainResponse> getServiceDomains()`
      - `public List<SupportCategoryResponse> getSupportCategories()`
      - `public SseEmitter streamCatalogEvents()`

`backend/src/main/java/com/smartcampus/maintenance/controller/ChatController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class ChatController`
    - Constructors:
      - `public ChatController(ChatService chatService, CurrentUserService currentUserService)`
    - Methods:
      - `public ChatMessageResponse sendMessage(Long ticketId, ChatSendRequest request)`
      - `public List<ChatMessageResponse> getMessages(Long ticketId)`

`backend/src/main/java/com/smartcampus/maintenance/controller/NotificationController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class NotificationController`
    - Constructors:
      - `public NotificationController(NotificationService notificationService, CurrentUserService currentUserService)`
    - Methods:
      - `public List<NotificationResponse> getNotifications()`
      - `public Map<String, Long> getUnreadCount()`
      - `public void markAllRead()`
      - `public void markRead(Long id)`

`backend/src/main/java/com/smartcampus/maintenance/controller/PublicSupportController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class PublicSupportController`
    - Constructors:
      - `public PublicSupportController(SupportRequestService supportRequestService, PublicEndpointSecurityService

publicEndpointSecurityService, RequestMetadataResolver requestMetadataResolver)`
  - Methods:
    - `public SupportContactResponse submitSupportRequest(SupportContactRequest request, HttpServletRequest servletRequest)`

`backend/src/main/java/com/smartcampus/maintenance/controller/TicketController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class TicketController`
    - Constructors:
      - `public TicketController(TicketService ticketService, CurrentUserService currentUserService)`
    - Methods:
      - `public CommentResponse addComment(Long id, CommentCreateRequest request)`
      - `public DuplicateCheckResponse checkDuplicates(TicketCreateRequest request)`
      - `public List<CommentResponse> getComments(Long id)`
      - `public List<TicketAssignmentRecommendationResponse> getAssignmentRecommendations(Long id)`
      - `public List<TicketLogResponse> getTicketLogs(Long id)`
      - `public List<TicketResponse> getAllTickets(TicketStatus status, String serviceDomainKey, Long requestTypeId, Long buildingId, UrgencyLevel urgency, Long assigneeId, String search)`
      - `public List<TicketResponse> getAssignedTickets()`
      - `public List<TicketResponse> getMyTickets()`
        - `public ResponseEntity<Resource> getAttachment(Long id, String attachmentType, Long expires, String signature)`
      - `public TicketDetailResponse getTicket(Long id)`
      - `public TicketRatingResponse rateTicket(Long id, TicketRateRequest request)`
      - `public TicketResponse assignTicket(Long id, TicketAssignRequest request)`
      - `public TicketResponse createTicket(TicketCreateRequest request)`
      - `public TicketResponse createTicketWithImage(TicketCreateRequest request, MultipartFile image)`
      - `public TicketResponse respondToAssignment(Long id, TicketAssignmentResponseRequest request)`
      - `public TicketResponse updateStatus(Long id, TicketStatusUpdateRequest request)`
      - `public TicketResponse uploadAfterPhoto(Long id, MultipartFile image)`

`backend/src/main/java/com/smartcampus/maintenance/controller/UserController.java`

* Language: `java`
* Purpose: API/controller logic
* Package: `com.smartcampus.maintenance.controller`
* Types and Methods:
  - `class UserController`
    - Constructors:
      - `public UserController(UserService userService, ScheduledBroadcastService scheduledBroadcastService, CurrentUserService currentUserService)`
    - Methods:
      - `public BroadcastMessageResponse broadcastMessage(BroadcastMessageRequest request)`
      - `public List<ScheduledBroadcastResponse> getScheduledBroadcasts()`
      - `public List<UserSummaryResponse> getMaintenanceUsers()`
      - `public List<UserWithTicketCountResponse> getUsers()`
      - `public ScheduledBroadcastResponse cancelScheduledBroadcast(Long id)`
      - `public ScheduledBroadcastResponse scheduleBroadcast(ScheduledBroadcastCreateRequest request)`
      - `public StaffInviteResponse createStaff(StaffInviteRequest request)`
      - `public UserProfileResponse getMyProfile()`
      - `public UserProfileResponse updateMyProfile(UserProfileUpdateRequest request)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/AnalyticsSummaryResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record AnalyticsSummaryResponse(long totalTickets, Map<String, Long> byStatus, Map<String, Long> byCategory,
Map<String, Long> byUrgency)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/CategoryResolutionTimeResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record CategoryResolutionTimeResponse(String category, double averageHours, long resolvedTickets)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/CrewPerformanceResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record CrewPerformanceResponse(Long userId, String username, String fullName, long resolvedTickets)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/DailyResolvedPointResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record DailyResolvedPointResponse(String date, long resolvedTickets)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/PublicLandingConfigResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record PublicLandingConfigResponse(String supportHours, String supportPhone, String supportTimezone, int urgentSlaHours, int standardSlaHours)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/PublicLandingStatsResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.

* Types and Methods:
  - `type unknown`
    - Methods:
      - `record PublicLandingStatsResponse(long totalTickets, long resolvedTickets, long openTickets, long resolvedToday, double averageResolutionHours, List<DailyResolvedPointResponse> resolvedLast7Days, LocalDateTime lastUpdatedAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/ResolutionTimeResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record ResolutionTimeResponse(double overallAverageHours, List<CategoryResolutionTimeResponse> byCategory)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/SlaComplianceResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record SlaComplianceResponse(long totalTickets, long onTimeTickets, long breachedTickets, double compliancePercentage, double avgResolutionHours, long criticalSlaHours, long highSlaHours, long mediumSlaHours, long lowSlaHours)`

`backend/src/main/java/com/smartcampus/maintenance/dto/analytics/TopBuildingResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record TopBuildingResponse(String building, long totalIssues)`

`backend/src/main/java/com/smartcampus/maintenance/dto/announcement/AnnouncementCreateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/announcement/AnnouncementResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:

- `record AnnouncementResponse(Long id, String title, String content, boolean active, String createdByUsername, String createdByFullName, LocalDateTime createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/AcceptStaffInviteRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/AuthResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record AuthResponse(String accessToken, String expiresAt, String username, String fullName, String role, Boolean mfaRequired, String mfaChallengeId, String message)`

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/CurrentUserResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record CurrentUserResponse(String username, String fullName, String role)`

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/ForgotPasswordRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/LoginRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/RegisterRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/ResendMfaRequest.java`

* Language: `java`
* Purpose: Data transfer model

* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/ResendVerificationRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/ResetPasswordRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/UsernameSuggestionsResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record UsernameSuggestionsResponse(String requestedUsername, List<String> suggestions)`

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/VerifyEmailRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/auth/VerifyMfaRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/building/BuildingCreateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/building/BuildingResponse.java`

* Language: `java`

* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record BuildingResponse(Long id, String name, String code, int floors, boolean active, int sortOrder, LocalDateTime createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/building/BuildingUpdateRequest.java`
* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/catalog/CatalogStreamEvent.java`
* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record CatalogStreamEvent(String resource, long version, String action, List<Long> changedIds, LocalDateTime changedAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/catalog/RequestTypeCreateRequest.java`
* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/catalog/RequestTypeResponse.java`
* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record RequestTypeResponse(Long id, String label, boolean active, int sortOrder, Long serviceDomainId, String serviceDomainKey, String serviceDomainLabel)`

`backend/src/main/java/com/smartcampus/maintenance/dto/catalog/RequestTypeUpdateRequest.java`
* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/catalog/ServiceDomainResponse.java`
* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.

* Types and Methods:
  - `type unknown`
    - Methods:
      - `record ServiceDomainResponse(Long id, String key, String label, int sortOrder)`

`backend/src/main/java/com/smartcampus/maintenance/dto/catalog/SupportCategoryCreateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/catalog/SupportCategoryResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record SupportCategoryResponse(Long id, String label, boolean active, int sortOrder)`

`backend/src/main/java/com/smartcampus/maintenance/dto/catalog/SupportCategoryUpdateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/chat/ChatMessageResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record ChatMessageResponse(Long id, Long ticketId, String senderUsername, String senderFullName, String senderRole, String content, LocalDateTime createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/chat/ChatSendRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/notification/NotificationResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:

- `record NotificationResponse(Long id, String title, String message, String type, boolean read, String linkUrl, LocalDateTime createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/support/SupportContactRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/support/SupportContactResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record SupportContactResponse(Long requestId, String message)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/CommentCreateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/CommentResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record CommentResponse(Long id, Long ticketId, String authorUsername, String authorFullName, String authorRole,
String content, LocalDateTime createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/DuplicateCheckResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record DuplicateCheckResponse(boolean hasSimilar, List<SimilarTicketSummary> similarTickets, String message)`
      - `record SimilarTicketSummary(Long id, String title, String status, String building, String category)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketAssignRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketAssignmentRecommendationResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record TicketAssignmentRecommendationResponse(Long userId, String username, String fullName, double score,
List<String> reasons)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketAssignmentResponseRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketBuildingResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record TicketBuildingResponse(Long id, String name, String code)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketCreateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketDetailResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record TicketDetailResponse(TicketResponse ticket, List<TicketLogResponse> logs, TicketRatingResponse rating)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketLogResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record TicketLogResponse(Long id, String oldStatus, String newStatus, String note, TicketUserInfoResponse

changedBy, LocalDateTime timestamp)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketRateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketRatingResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record TicketRatingResponse(Integer stars, String comment, TicketUserInfoResponse ratedBy, LocalDateTime createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketRequestTypeResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record TicketRequestTypeResponse(Long id, String label)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record TicketResponse(Long id, String title, String description, String category, String serviceDomainKey, TicketRequestTypeResponse requestType, TicketBuildingResponse building, String location, String urgency, String status,
TicketUserInfoResponse createdBy, TicketUserInfoResponse assignedTo, String imageUrl, String afterImageUrl, LocalDateTime createdAt, LocalDateTime updatedAt, LocalDateTime resolvedAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketStatusUpdateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/ticket/TicketUserInfoResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:

- `record TicketUserInfoResponse(Long id, String username, String fullName, String role)`

`backend/src/main/java/com/smartcampus/maintenance/dto/user/BroadcastAudience.java`

* Language: `java`
* Purpose: Data transfer model
* Package: `com.smartcampus.maintenance.dto.user`
* Types and Methods:
  - `enum BroadcastAudience`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/user/BroadcastMessageRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/user/BroadcastMessageResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record BroadcastMessageResponse(String title, String audience, int recipientCount, LocalDateTime createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/user/CreateStaffRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/user/ScheduledBroadcastCreateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/user/ScheduledBroadcastResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record ScheduledBroadcastResponse(Long id, String title, String message, String audience, String status, LocalDateTime scheduledFor, int recipientCount, LocalDateTime createdAt, LocalDateTime sentAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/user/StaffInviteRequest.java`

* Language: `java`
* Purpose: Data transfer model

* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/user/StaffInviteResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record StaffInviteResponse(Long id, String email, String fullName, LocalDateTime expiresAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/user/UserProfileResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record UserProfileResponse(Long id, String username, String email, String fullName, String role, LocalDateTime createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/user/UserProfileUpdateRequest.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/dto/user/UserSummaryResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record UserSummaryResponse(Long id, String username, String email, String fullName, String role, LocalDateTime
createdAt)`

`backend/src/main/java/com/smartcampus/maintenance/dto/user/UserWithTicketCountResponse.java`

* Language: `java`
* Purpose: Data transfer model
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record UserWithTicketCountResponse(Long id, String username, String email, String fullName, String role, long ticketCount)`

`backend/src/main/java/com/smartcampus/maintenance/entity/Announcement.java`

* Language: `java`
* Purpose: Domain/entity model

* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class Announcement`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public String getContent()`
      - `public String getTitle()`
      - `public User getCreatedBy()`
      - `public boolean isActive()`
      - `public void onCreate()`
      - `public void setActive(boolean active)`
      - `public void setContent(String content)`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setCreatedBy(User createdBy)`
      - `public void setId(Long id)`
      - `public void setTitle(String title)`

`backend/src/main/java/com/smartcampus/maintenance/entity/AuditEvent.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class AuditEvent`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public String getAction()`
      - `public String getActorRole()`
      - `public String getActorUsername()`
      - `public String getDetailsJson()`
      - `public String getIpAddress()`
      - `public String getTargetId()`
      - `public String getTargetType()`
      - `public String getUserAgent()`
      - `public User getActorUser()`
      - `public void onCreate()`
      - `public void setAction(String action)`
      - `public void setActorRole(String actorRole)`
      - `public void setActorUser(User actorUser)`
      - `public void setActorUsername(String actorUsername)`
      - `public void setDetailsJson(String detailsJson)`
      - `public void setId(Long id)`
      - `public void setIpAddress(String ipAddress)`
      - `public void setTargetId(String targetId)`
      - `public void setTargetType(String targetType)`
      - `public void setUserAgent(String userAgent)`

`backend/src/main/java/com/smartcampus/maintenance/entity/AuthMfaChallenge.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class AuthMfaChallenge`
    - Methods:

- `public LocalDateTime getCreatedAt()`
- `public LocalDateTime getExpiresAt()`
- `public LocalDateTime getResendAvailableAt()`
- `public Long getId()`
- `public String getChallengeId()`
- `public String getCodeHash()`
- `public User getUser()`
- `public boolean isConsumed()`
- `public boolean isExpired()`
- `public int getAttemptCount()`
- `public void onCreate()`
- `public void setAttemptCount(int attemptCount)`
- `public void setChallengeId(String challengeId)`
- `public void setCodeHash(String codeHash)`
- `public void setConsumed(boolean consumed)`
- `public void setExpiresAt(LocalDateTime expiresAt)`
- `public void setResendAvailableAt(LocalDateTime resendAvailableAt)`
- `public void setUser(User user)`

`backend/src/main/java/com/smartcampus/maintenance/entity/AuthRefreshToken.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class AuthRefreshToken`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public LocalDateTime getExpiresAt()`
      - `public LocalDateTime getLastUsedAt()`
      - `public LocalDateTime getRevokedAt()`
      - `public LocalDateTime getRotatedAt()`
      - `public Long getId()`
      - `public String getIpAddress()`
      - `public String getReplacedByTokenHash()`
      - `public String getTokenFamily()`
      - `public String getTokenHash()`
      - `public String getUserAgent()`
      - `public User getUser()`
      - `public boolean isExpired()`
      - `public boolean isRevoked()`
      - `public void onCreate()`
      - `public void setExpiresAt(LocalDateTime expiresAt)`
      - `public void setId(Long id)`
      - `public void setIpAddress(String ipAddress)`
      - `public void setLastUsedAt(LocalDateTime lastUsedAt)`
      - `public void setReplacedByTokenHash(String replacedByTokenHash)`
      - `public void setRevokedAt(LocalDateTime revokedAt)`
      - `public void setRotatedAt(LocalDateTime rotatedAt)`
      - `public void setTokenFamily(String tokenFamily)`
      - `public void setTokenHash(String tokenHash)`
      - `public void setUser(User user)`
      - `public void setUserAgent(String userAgent)`

`backend/src/main/java/com/smartcampus/maintenance/entity/Building.java`

* Language: `java`

* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class Building`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public String getCode()`
      - `public String getName()`
      - `public boolean isActive()`
      - `public int getFloors()`
      - `public int getSortOrder()`
      - `public void onCreate()`
      - `public void setActive(boolean active)`
      - `public void setCode(String code)`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setFloors(int floors)`
      - `public void setId(Long id)`
      - `public void setName(String name)`
      - `public void setSortOrder(int sortOrder)`

`backend/src/main/java/com/smartcampus/maintenance/entity/ChatMessage.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class ChatMessage`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public String getContent()`
      - `public Ticket getTicket()`
      - `public User getSender()`
      - `public void onCreate()`
      - `public void setContent(String content)`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setId(Long id)`
      - `public void setSender(User sender)`
      - `public void setTicket(Ticket ticket)`

`backend/src/main/java/com/smartcampus/maintenance/entity/EmailOutbox.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class EmailOutbox`
    - Methods:
      - `public EmailOutboxStatus getStatus()`
      - `public LocalDateTime getCreatedAt()`
      - `public LocalDateTime getLastAttemptAt()`
      - `public LocalDateTime getNextAttemptAt()`
      - `public LocalDateTime getSentAt()`
      - `public Long getId()`
      - `public String getHtmlBody()`
      - `public String getLastError()`

- `public String getPlainTextBody()`
- `public String getSubject()`
- `public String getToEmail()`
- `public int getAttemptCount()`
- `public void onCreate()`
- `public void setAttemptCount(int attemptCount)`
- `public void setCreatedAt(LocalDateTime createdAt)`
- `public void setHtmlBody(String htmlBody)`
- `public void setId(Long id)`
- `public void setLastAttemptAt(LocalDateTime lastAttemptAt)`
- `public void setLastError(String lastError)`
- `public void setNextAttemptAt(LocalDateTime nextAttemptAt)`
- `public void setPlainTextBody(String plainTextBody)`
- `public void setSentAt(LocalDateTime sentAt)`
- `public void setStatus(EmailOutboxStatus status)`
- `public void setSubject(String subject)`
- `public void setToEmail(String toEmail)`

`backend/src/main/java/com/smartcampus/maintenance/entity/EmailVerificationToken.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class EmailVerificationToken`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public LocalDateTime getExpiresAt()`
      - `public Long getId()`
      - `public String getCode()`
      - `public User getUser()`
      - `public boolean isExpired()`
      - `public boolean isUsed()`
      - `public int getAttemptCount()`
      - `public void onCreate()`
      - `public void setAttemptCount(int attemptCount)`
      - `public void setCode(String code)`
      - `public void setExpiresAt(LocalDateTime expiresAt)`
      - `public void setUsed(boolean used)`
      - `public void setUser(User user)`

`backend/src/main/java/com/smartcampus/maintenance/entity/Notification.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class Notification`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public NotificationType getType()`
      - `public String getLinkUrl()`
      - `public String getMessage()`
      - `public String getTitle()`
      - `public User getUser()`
      - `public boolean isRead()`

- `public void onCreate()`
- `public void setCreatedAt(LocalDateTime createdAt)`
- `public void setId(Long id)`
- `public void setLinkUrl(String linkUrl)`
- `public void setMessage(String message)`
- `public void setRead(boolean read)`
- `public void setTitle(String title)`
- `public void setType(NotificationType type)`
- `public void setUser(User user)`

`backend/src/main/java/com/smartcampus/maintenance/entity/PasswordResetToken.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class PasswordResetToken`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public LocalDateTime getExpiresAt()`
      - `public Long getId()`
      - `public String getToken()`
      - `public User getUser()`
      - `public boolean isExpired()`
      - `public boolean isUsed()`
      - `public void onCreate()`
      - `public void setExpiresAt(LocalDateTime expiresAt)`
      - `public void setId(Long id)`
      - `public void setToken(String token)`
      - `public void setUsed(boolean used)`
      - `public void setUser(User user)`

`backend/src/main/java/com/smartcampus/maintenance/entity/PendingRegistration.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class PendingRegistration`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public LocalDateTime getLastVerificationSentAt()`
      - `public LocalDateTime getResendAvailableAt()`
      - `public LocalDateTime getUpdatedAt()`
      - `public LocalDateTime getVerificationTokenExpiresAt()`
      - `public Long getId()`
      - `public String getEmail()`
      - `public String getFullName()`
      - `public String getPasswordHash()`
      - `public String getUsername()`
      - `public String getVerificationTokenHash()`
      - `public boolean isVerificationTokenExpired()`
      - `public void onCreate()`
      - `public void onUpdate()`
      - `public void setEmail(String email)`
      - `public void setFullName(String fullName)`
      - `public void setId(Long id)`

    - `public void setLastVerificationSentAt(LocalDateTime lastVerificationSentAt)`
    - `public void setPasswordHash(String passwordHash)`
    - `public void setResendAvailableAt(LocalDateTime resendAvailableAt)`
    - `public void setUsername(String username)`
    - `public void setVerificationTokenExpiresAt(LocalDateTime verificationTokenExpiresAt)`
    - `public void setVerificationTokenHash(String verificationTokenHash)`

`backend/src/main/java/com/smartcampus/maintenance/entity/RequestType.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class RequestType`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public ServiceDomain getServiceDomain()`
      - `public String getLabel()`
      - `public boolean isActive()`
      - `public int getSortOrder()`
      - `public void onCreate()`
      - `public void setActive(boolean active)`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setId(Long id)`
      - `public void setLabel(String label)`
      - `public void setServiceDomain(ServiceDomain serviceDomain)`
      - `public void setSortOrder(int sortOrder)`

`backend/src/main/java/com/smartcampus/maintenance/entity/ScheduledBroadcast.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class ScheduledBroadcast`
    - Methods:
      - `public BroadcastAudience getAudience()`
      - `public LocalDateTime getCreatedAt()`
      - `public LocalDateTime getScheduledFor()`
      - `public LocalDateTime getSentAt()`
      - `public Long getId()`
      - `public ScheduledBroadcastStatus getStatus()`
      - `public String getMessage()`
      - `public String getTitle()`
      - `public User getCreatedBy()`
      - `public int getRecipientCount()`
      - `public void onCreate()`
      - `public void setAudience(BroadcastAudience audience)`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setCreatedBy(User createdBy)`
      - `public void setId(Long id)`
      - `public void setMessage(String message)`
      - `public void setRecipientCount(int recipientCount)`
      - `public void setScheduledFor(LocalDateTime scheduledFor)`
      - `public void setSentAt(LocalDateTime sentAt)`
      - `public void setStatus(ScheduledBroadcastStatus status)`

- `public void setTitle(String title)`

`backend/src/main/java/com/smartcampus/maintenance/entity/ServiceDomain.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class ServiceDomain`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public String getKey()`
      - `public String getLabel()`
      - `public int getSortOrder()`
      - `public void onCreate()`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setId(Long id)`
      - `public void setKey(String key)`
      - `public void setLabel(String label)`
      - `public void setSortOrder(int sortOrder)`

`backend/src/main/java/com/smartcampus/maintenance/entity/StaffInvite.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class StaffInvite`
    - Methods:
      - `public LocalDateTime getAcceptedAt()`
      - `public LocalDateTime getCreatedAt()`
      - `public LocalDateTime getExpiresAt()`
      - `public Long getId()`
      - `public String getEmail()`
      - `public String getFullName()`
      - `public String getTokenHash()`
      - `public String getUsername()`
      - `public User getInvitedBy()`
      - `public boolean isExpired()`
      - `public boolean isUsed()`
      - `public void onCreate()`
      - `public void setAcceptedAt(LocalDateTime acceptedAt)`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setEmail(String email)`
      - `public void setExpiresAt(LocalDateTime expiresAt)`
      - `public void setFullName(String fullName)`
      - `public void setId(Long id)`
      - `public void setInvitedBy(User invitedBy)`
      - `public void setTokenHash(String tokenHash)`
      - `public void setUsed(boolean used)`
      - `public void setUsername(String username)`

`backend/src/main/java/com/smartcampus/maintenance/entity/SupportCategory.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:

- `class SupportCategory`
  - Methods:
    - `public LocalDateTime getCreatedAt()`
    - `public Long getId()`
    - `public String getLabel()`
    - `public boolean isActive()`
    - `public int getSortOrder()`
    - `public void onCreate()`
    - `public void setActive(boolean active)`
    - `public void setCreatedAt(LocalDateTime createdAt)`
    - `public void setId(Long id)`
    - `public void setLabel(String label)`
    - `public void setSortOrder(int sortOrder)`

`backend/src/main/java/com/smartcampus/maintenance/entity/SupportRequest.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class SupportRequest`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public String getCategory()`
      - `public String getEmail()`
      - `public String getFullName()`
      - `public String getMessage()`
      - `public String getSubject()`
      - `public SupportCategory getSupportCategory()`
      - `public void onCreate()`
      - `public void setCategory(String category)`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setEmail(String email)`
      - `public void setFullName(String fullName)`
      - `public void setId(Long id)`
      - `public void setMessage(String message)`
      - `public void setSubject(String subject)`
      - `public void setSupportCategory(SupportCategory supportCategory)`

`backend/src/main/java/com/smartcampus/maintenance/entity/Ticket.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class Ticket`
    - Methods:
      - `public Building getBuildingRecord()`
      - `public LocalDateTime getCreatedAt()`
      - `public LocalDateTime getResolvedAt()`
      - `public LocalDateTime getUpdatedAt()`
      - `public Long getId()`
      - `public RequestType getRequestType()`
      - `public String getAfterImagePath()`
      - `public String getBuilding()`
      - `public String getDescription()`

- `public String getImagePath()`
- `public String getLocation()`
- `public String getTitle()`
- `public TicketCategory getCategory()`
- `public TicketStatus getStatus()`
- `public UrgencyLevel getUrgency()`
- `public User getAssignedTo()`
- `public User getCreatedBy()`
- `public void onCreate()`
- `public void onUpdate()`
- `public void setAfterImagePath(String afterImagePath)`
- `public void setAssignedTo(User assignedTo)`
- `public void setBuilding(String building)`
- `public void setBuildingRecord(Building buildingRecord)`
- `public void setCategory(TicketCategory category)`
- `public void setCreatedAt(LocalDateTime createdAt)`
- `public void setCreatedBy(User createdBy)`
- `public void setDescription(String description)`
- `public void setId(Long id)`
- `public void setImagePath(String imagePath)`
- `public void setLocation(String location)`
- `public void setRequestType(RequestType requestType)`
- `public void setResolvedAt(LocalDateTime resolvedAt)`
- `public void setStatus(TicketStatus status)`
- `public void setTitle(String title)`
- `public void setUpdatedAt(LocalDateTime updatedAt)`
- `public void setUrgency(UrgencyLevel urgency)`

`backend/src/main/java/com/smartcampus/maintenance/entity/TicketComment.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class TicketComment`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public String getContent()`
      - `public Ticket getTicket()`
      - `public User getAuthor()`
      - `public void onCreate()`
      - `public void setAuthor(User author)`
      - `public void setContent(String content)`
      - `public void setCreatedAt(LocalDateTime createdAt)`
      - `public void setId(Long id)`
      - `public void setTicket(Ticket ticket)`

`backend/src/main/java/com/smartcampus/maintenance/entity/TicketLog.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class TicketLog`
    - Methods:
      - `public LocalDateTime getTimestamp()`

    - `public Long getId()`
    - `public String getNote()`
    - `public Ticket getTicket()`
    - `public TicketStatus getNewStatus()`
    - `public TicketStatus getOldStatus()`
    - `public User getChangedBy()`
    - `public void onCreate()`
    - `public void setChangedBy(User changedBy)`
    - `public void setNewStatus(TicketStatus newStatus)`
    - `public void setNote(String note)`
    - `public void setOldStatus(TicketStatus oldStatus)`
    - `public void setTicket(Ticket ticket)`

`backend/src/main/java/com/smartcampus/maintenance/entity/TicketRating.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class TicketRating`
    - Methods:
      - `public Integer getStars()`
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public String getComment()`
      - `public Ticket getTicket()`
      - `public User getRatedBy()`
      - `public void onCreate()`
      - `public void setComment(String comment)`
      - `public void setRatedBy(User ratedBy)`
      - `public void setStars(Integer stars)`
      - `public void setTicket(Ticket ticket)`

`backend/src/main/java/com/smartcampus/maintenance/entity/User.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity`
* Types and Methods:
  - `class User`
    - Methods:
      - `public LocalDateTime getCreatedAt()`
      - `public Long getId()`
      - `public Role getRole()`
      - `public String getEmail()`
      - `public String getFullName()`
      - `public String getPasswordHash()`
      - `public String getUsername()`
      - `public boolean isEmailVerified()`
      - `public boolean isMfaEnabled()`
      - `public int getTokenVersion()`
      - `public void onCreate()`
      - `public void setEmail(String email)`
      - `public void setEmailVerified(boolean emailVerified)`
      - `public void setFullName(String fullName)`
      - `public void setId(Long id)`
      - `public void setMfaEnabled(boolean mfaEnabled)`

- `public void setPasswordHash(String passwordHash)`
- `public void setRole(Role role)`
- `public void setTokenVersion(int tokenVersion)`
- `public void setUsername(String username)`

`backend/src/main/java/com/smartcampus/maintenance/entity/enums/EmailOutboxStatus.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity.enums`
* Types and Methods:
  - `enum EmailOutboxStatus`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/entity/enums/NotificationType.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity.enums`
* Types and Methods:
  - `enum NotificationType`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/entity/enums/Role.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity.enums`
* Types and Methods:
  - `enum Role`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/entity/enums/ScheduledBroadcastStatus.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity.enums`
* Types and Methods:
  - `enum ScheduledBroadcastStatus`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/entity/enums/TicketCategory.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity.enums`
* Types and Methods:
  - `enum TicketCategory`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/entity/enums/TicketStatus.java`

* Language: `java`
* Purpose: Domain/entity model
* Package: `com.smartcampus.maintenance.entity.enums`
* Types and Methods:
  - `enum TicketStatus`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/entity/enums/UrgencyLevel.java`

* Language: `java`
* Purpose: Domain/entity model

* Package: `com.smartcampus.maintenance.entity.enums`
* Types and Methods:
  - `enum UrgencyLevel`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/event/PendingRegistrationVerificationRequestedEvent.java`

* Language: `java`
* Purpose: Project source code
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record PendingRegistrationVerificationRequestedEvent(String email, String fullName, String verificationCode, long expiresInMinutes)`

`backend/src/main/java/com/smartcampus/maintenance/event/PendingRegistrationVerifiedEvent.java`

* Language: `java`
* Purpose: Project source code
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record PendingRegistrationVerifiedEvent(String email, String fullName, String loginUrl)`

`backend/src/main/java/com/smartcampus/maintenance/exception/ApiException.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.exception`
* Types and Methods:
  - `class ApiException`
    - Constructors:
      - `public ApiException(HttpStatus status, String message)`
    - Methods:
      - `public HttpStatus getStatus()`

`backend/src/main/java/com/smartcampus/maintenance/exception/BadRequestException.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.exception`
* Types and Methods:
  - `class BadRequestException`
    - Constructors:
      - `public BadRequestException(String message)`

`backend/src/main/java/com/smartcampus/maintenance/exception/ConflictException.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.exception`
* Types and Methods:
  - `class ConflictException`
    - Constructors:
      - `public ConflictException(String message)`

`backend/src/main/java/com/smartcampus/maintenance/exception/ErrorResponse.java`

* Language: `java`
* Purpose: Data transfer model

* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record ErrorResponse(LocalDateTime timestamp, int status, String error, String message, String path)`

`backend/src/main/java/com/smartcampus/maintenance/exception/ForbiddenException.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.exception`
* Types and Methods:
  - `class ForbiddenException`
    - Constructors:
      - `public ForbiddenException(String message)`

`backend/src/main/java/com/smartcampus/maintenance/exception/GlobalExceptionHandler.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.exception`
* Types and Methods:
  - `class GlobalExceptionHandler`
    - Methods:
      - `private ResponseEntity<ErrorResponse> build(HttpStatus status, String message, String path)`
      - `private String formatFieldError(FieldError fieldError)`
        - `public ResponseEntity<ErrorResponse> handleAccessDenied(AccessDeniedException ex, HttpServletRequest request)`
      - `public ResponseEntity<ErrorResponse> handleApiException(ApiException ex, HttpServletRequest request)`
      - `public ResponseEntity<ErrorResponse> handleUnexpected(Exception ex, HttpServletRequest request)`
        - `public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex, HttpServletRequest
request)`

`backend/src/main/java/com/smartcampus/maintenance/exception/NotFoundException.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.exception`
* Types and Methods:
  - `class NotFoundException`
    - Constructors:
      - `public NotFoundException(String message)`

`backend/src/main/java/com/smartcampus/maintenance/exception/UnauthorizedException.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.exception`
* Types and Methods:
  - `class UnauthorizedException`
    - Constructors:
      - `public UnauthorizedException(String message)`

`backend/src/main/java/com/smartcampus/maintenance/exception/UnprocessableEntityException.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.exception`
* Types and Methods:

- `class UnprocessableEntityException`
  - Constructors:
    - `public UnprocessableEntityException(String message)`

`backend/src/main/java/com/smartcampus/maintenance/mapper/TicketMapper.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.mapper`
* Types and Methods:
  - `class TicketMapper`
    - Constructors:
      - `private TicketMapper()`
    - Methods:
      - `private static TicketBuildingResponse toBuildingResponse(Building building, String legacyBuildingName)`
      - `private static TicketRequestTypeResponse toRequestTypeResponse(RequestType requestType)`
      - `public static String resolveServiceDomainKey(Ticket ticket)`
      - `public static TicketLogResponse toLogResponse(TicketLog log)`
      - `public static TicketRatingResponse toRatingResponse(TicketRating rating)`
      - `public static TicketResponse toResponse(Ticket ticket, String imageUrl, String afterImageUrl)`

`backend/src/main/java/com/smartcampus/maintenance/mapper/UserMapper.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.mapper`
* Types and Methods:
  - `class UserMapper`
    - Constructors:
      - `private UserMapper()`
    - Methods:
      - `public static TicketUserInfoResponse toTicketUserInfo(User user)`
      - `public static UserSummaryResponse toSummary(User user)`
      - `public static UserWithTicketCountResponse toWithTicketCount(User user, long ticketCount)`

`backend/src/main/java/com/smartcampus/maintenance/optimization/AssignmentCandidateMetrics.java`

* Language: `java`
* Purpose: Project source code
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record AssignmentCandidateMetrics(Long userId, String username, String fullName, int activeOpenTickets, int sameDomainResolvedTickets, int sameBuildingResolvedTickets, int recentResolvedTickets)`

`backend/src/main/java/com/smartcampus/maintenance/optimization/AssignmentScorer.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.optimization`
* Types and Methods:
  - `class AssignmentScorer`
    - Constructors:
      - `private AssignmentScorer()`
    - Methods:
      - `public static double scoreCandidate(AssignmentCandidateMetrics candidate)`

`backend/src/main/java/com/smartcampus/maintenance/optimization/JavaImageOptimizer.java`

* Language: `java`
* Purpose: Project source code
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class JavaImageOptimizer`
    - Methods:
      - ` try(ByteArrayOutputStream outputBytes = new ByteArrayOutputStream()`
      - `BufferedImage ensureRgb(BufferedImage source)`
      - `Optional<BufferedImage> decode(byte[] imageBytes)`
      - `Optional<OptimizedImageResult> optimize(String contentType, byte[] originalBytes, int minSavingsPercent, int jpegQuality, int pngCompressionQuality)`
      - `Optional<byte[]> encodeJpeg(BufferedImage source, int quality)`
      - `Optional<byte[]> encodePng(BufferedImage source, int quality)`
      - `Optional<byte[]> encodeWithWriter(BufferedImage source, String formatName, int quality)`
      - `String normalizeContentType(String contentType)`
      - `int clampQuality(int quality)`
      - `private JavaImageOptimizer()`
      - `record OptimizedImageResult(byte[] bytes, String contentType, int originalWidth, int originalHeight, int optimizedWidth, int optimizedHeight)`
      - `return encodeWithWriter(rgbSource, "jpeg", quality)`
      - `return encodeWithWriter(source, "png", quality)`

`backend/src/main/java/com/smartcampus/maintenance/repository/AnnouncementRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface AnnouncementRepository`
    - Methods:
      - `List<Announcement> findByActiveTrueOrderByCreatedAtDesc()`

`backend/src/main/java/com/smartcampus/maintenance/repository/AuditEventRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface AuditEventRepository`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/repository/AuthMfaChallengeRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface AuthMfaChallengeRepository`
    - Methods:
      - `Optional<AuthMfaChallenge> findByChallengeIdAndConsumedFalse(String challengeId)`
      - `long deleteByConsumedTrueAndCreatedAtBefore(LocalDateTime cutoff)`
      - `long deleteByExpiresAtBefore(LocalDateTime cutoff)`

`backend/src/main/java/com/smartcampus/maintenance/repository/AuthRefreshTokenRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`

* Types and Methods:
  - `interface AuthRefreshTokenRepository`
    - Methods:
      - `List<AuthRefreshToken> findByTokenFamilyAndRevokedAtIsNull(String tokenFamily)`
      - `List<AuthRefreshToken> findByUser_IdAndRevokedAtIsNull(Long userId)`
      - `Optional<AuthRefreshToken> findByTokenHash(String tokenHash)`
      - `long deleteByExpiresAtBefore(LocalDateTime expiresAt)`
      - `long deleteByRevokedAtBefore(LocalDateTime revokedAt)`

`backend/src/main/java/com/smartcampus/maintenance/repository/BuildingRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface BuildingRepository`
    - Methods:
      - `List<Building> findAllByOrderBySortOrderAscNameAsc()`
      - `List<Building> findByActiveTrueOrderBySortOrderAscNameAsc()`
      - `Optional<Building> findByNameIgnoreCase(String name)`
      - `boolean existsByCodeIgnoreCase(String code)`
      - `boolean existsByCodeIgnoreCaseAndIdNot(String code, Long id)`
      - `boolean existsByNameIgnoreCase(String name)`
      - `boolean existsByNameIgnoreCaseAndIdNot(String name, Long id)`

`backend/src/main/java/com/smartcampus/maintenance/repository/ChatMessageRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface ChatMessageRepository`
    - Methods:
      - `List<ChatMessage> findByTicketIdOrderByCreatedAtAsc(Long ticketId)`

`backend/src/main/java/com/smartcampus/maintenance/repository/EmailOutboxRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface EmailOutboxRepository`
    - Methods:
      - `List<EmailOutbox> findByStatusAndNextAttemptAtLessThanEqualOrderByCreatedAtAsc(EmailOutboxStatus status,
LocalDateTime now, Pageable pageable)`
      - `long deleteByStatusAndCreatedAtBefore(EmailOutboxStatus status, LocalDateTime cutoff)`

`backend/src/main/java/com/smartcampus/maintenance/repository/EmailVerificationTokenRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface EmailVerificationTokenRepository`
    - Methods:
      - `Optional<EmailVerificationToken> findByUser_IdAndCodeAndUsedFalse(Long userId, String code)`
      - `Optional<EmailVerificationToken> findTopByUser_IdAndUsedFalseOrderByCreatedAtDesc(Long userId)`
      - `long deleteByExpiresAtBefore(LocalDateTime expiresAt)`

- `long deleteByUsedTrueAndCreatedAtBefore(LocalDateTime createdAt)`
- `void deleteByUser_IdAndUsedFalse(Long userId)`

`backend/src/main/java/com/smartcampus/maintenance/repository/NotificationRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface NotificationRepository`
    - Methods:
      - `List<Notification> findByUserIdAndReadFalseOrderByCreatedAtDesc(Long userId)`
      - `List<Notification> findByUserIdOrderByCreatedAtDesc(Long userId)`
      - `long countByUserIdAndReadFalse(Long userId)`

`backend/src/main/java/com/smartcampus/maintenance/repository/PasswordResetTokenRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface PasswordResetTokenRepository`
    - Methods:
      - `Optional<PasswordResetToken> findByTokenAndUsedFalse(String token)`
      - `Optional<PasswordResetToken> findTopByUser_IdAndUsedFalseOrderByCreatedAtDesc(Long userId)`
      - `boolean existsByToken(String token)`
      - `long countByUser_IdAndUsedFalse(Long userId)`
      - `long deleteByExpiresAtBefore(LocalDateTime expiresAt)`
      - `long deleteByUsedTrueAndCreatedAtBefore(LocalDateTime createdAt)`
      - `void deleteByUser_IdAndUsedFalse(Long userId)`

`backend/src/main/java/com/smartcampus/maintenance/repository/PendingRegistrationRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface PendingRegistrationRepository`
    - Methods:
      - `Optional<PendingRegistration> findByEmailIgnoreCase(String email)`
      - `Optional<PendingRegistration> findByUsernameIgnoreCase(String username)`
      - `Optional<PendingRegistration> findByVerificationTokenHash(String verificationTokenHash)`
      - `boolean existsByEmailIgnoreCase(String email)`
      - `boolean existsByEmailIgnoreCaseAndIdNot(String email, Long id)`
      - `boolean existsByUsernameIgnoreCase(String username)`
      - `boolean existsByUsernameIgnoreCaseAndIdNot(String username, Long id)`

`backend/src/main/java/com/smartcampus/maintenance/repository/RequestTypeRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface RequestTypeRepository`
    - Methods:
      - `List<RequestType> findAllByOrderByServiceDomainSortOrderAscServiceDomainLabelAscSortOrderAscLabelAsc()`
      - `List<RequestType>

findByActiveTrueOrderByServiceDomainSortOrderAscServiceDomainLabelAscSortOrderAscLabelAsc()`
    - `List<RequestType> findByServiceDomain_KeyIgnoreCaseAndActiveTrueOrderBySortOrderAscLabelAsc(String serviceDomainKey)`
        - `Optional<RequestType> findFirstByServiceDomain_KeyIgnoreCaseOrderBySortOrderAscIdAsc(String serviceDomainKey)`
    - `boolean existsByServiceDomain_IdAndLabelIgnoreCase(Long serviceDomainId, String label)`
    - `boolean existsByServiceDomain_IdAndLabelIgnoreCaseAndIdNot(Long serviceDomainId, String label, Long id)`

`backend/src/main/java/com/smartcampus/maintenance/repository/ScheduledBroadcastRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface ScheduledBroadcastRepository`
    - Methods:
      - `List<ScheduledBroadcast> findAllByOrderByScheduledForAscCreatedAtAsc()`
      - `List<ScheduledBroadcast> findByStatusAndScheduledForLessThanEqualOrderByScheduledForAsc(ScheduledBroadcastStatus status, LocalDateTime scheduledFor)`

`backend/src/main/java/com/smartcampus/maintenance/repository/ServiceDomainRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface ServiceDomainRepository`
    - Methods:
      - `List<ServiceDomain> findAllByOrderBySortOrderAscLabelAsc()`
      - `Optional<ServiceDomain> findByKeyIgnoreCase(String key)`

`backend/src/main/java/com/smartcampus/maintenance/repository/StaffInviteRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface StaffInviteRepository`
    - Methods:
      - `Optional<StaffInvite> findByTokenHashAndUsedFalse(String tokenHash)`
      - `boolean existsByEmailIgnoreCaseAndUsedFalse(String email)`
      - `boolean existsByUsernameIgnoreCaseAndUsedFalse(String username)`
      - `long deleteByExpiresAtBefore(LocalDateTime cutoff)`
      - `long deleteByUsedTrueAndAcceptedAtBefore(LocalDateTime cutoff)`

`backend/src/main/java/com/smartcampus/maintenance/repository/SupportCategoryRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface SupportCategoryRepository`
    - Methods:
      - `List<SupportCategory> findAllByOrderBySortOrderAscLabelAsc()`
      - `List<SupportCategory> findByActiveTrueOrderBySortOrderAscLabelAsc()`
      - `boolean existsByLabelIgnoreCase(String label)`
      - `boolean existsByLabelIgnoreCaseAndIdNot(String label, Long id)`

`backend/src/main/java/com/smartcampus/maintenance/repository/SupportRequestRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface SupportRequestRepository`
    - No methods detected.

`backend/src/main/java/com/smartcampus/maintenance/repository/TicketCommentRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface TicketCommentRepository`
    - Methods:
      - `List<TicketComment> findByTicketIdOrderByCreatedAtAsc(Long ticketId)`
      - `long countByTicketId(Long ticketId)`

`backend/src/main/java/com/smartcampus/maintenance/repository/TicketLogRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface TicketLogRepository`
    - Methods:
      - `List<TicketLog> findByTicketIdOrderByTimestampAsc(Long ticketId)`

`backend/src/main/java/com/smartcampus/maintenance/repository/TicketRatingRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface TicketRatingRepository`
    - Methods:
      - `Optional<TicketRating> findByTicketId(Long ticketId)`
      - `boolean existsByTicketId(Long ticketId)`

`backend/src/main/java/com/smartcampus/maintenance/repository/TicketRepository.java`

* Language: `java`
* Purpose: Data access layer
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type interface TicketRepository`
    - Methods:
      - `List<Object[]> countByBuilding()`
      - `List<Ticket> findByAssignedToIdOrderByCreatedAtDesc(Long assignedToId)`
      - `List<Ticket> findByCreatedByIdOrderByCreatedAtDesc(Long createdById)`
      - `List<Ticket> findByRequestTypeIdAndBuildingRecordIdAndStatusNotIn(Long requestTypeId, Long buildingRecordId, Collection<TicketStatus> statuses)`
      - `List<Ticket> findByStatusAndAssignedToIsNullAndUpdatedAtBefore(TicketStatus status, LocalDateTime updatedAt)`
      - `long countByAssignedToId(Long assignedToId)`
      - `long countByAssignedToIdAndBuildingRecord_IdAndStatusIn(Long assignedToId, Long buildingRecordId, Collection<TicketStatus> statuses)`
      - `long countByAssignedToIdAndRequestType_ServiceDomain_KeyAndStatusIn(Long assignedToId, String

serviceDomainKey,
Collection<TicketStatus> statuses)`
- `long countByAssignedToIdAndResolvedAtAfterAndStatusIn(Long assignedToId, java.time.LocalDateTime resolvedAfter,
Collection<TicketStatus> statuses)`
- `long countByAssignedToIdAndStatusIn(Long assignedToId, Collection<TicketStatus> statuses)`
- `long countByCreatedById(Long createdById)`
- `long countByCreatedByIdAndStatusIn(Long createdById, Collection<TicketStatus> statuses)`

`backend/src/main/java/com/smartcampus/maintenance/repository/TicketSpecifications.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `class TicketSpecifications`
    - Constructors:
      - `private TicketSpecifications()`
    - Methods:
      - `public static Specification<Ticket> assigneeEquals(Long assigneeId)`
      - `public static Specification<Ticket> buildingEquals(Long buildingId)`
      - `public static Specification<Ticket> requestTypeEquals(Long requestTypeId)`
      - `public static Specification<Ticket> searchLike(String search)`
      - `public static Specification<Ticket> serviceDomainKeyEquals(String serviceDomainKey)`
      - `public static Specification<Ticket> statusEquals(TicketStatus status)`
      - `public static Specification<Ticket> urgencyEquals(UrgencyLevel urgency)`

`backend/src/main/java/com/smartcampus/maintenance/repository/UserRepository.java`

* Language: `java`
* Purpose: Data access layer
* Package: `com.smartcampus.maintenance.repository`
* Types and Methods:
  - `interface UserRepository`
    - Methods:
      - `List<User> findByRole(Role role)`
      - `List<User> findByRoleOrderByFullNameAsc(Role role)`
      - `Optional<User> findByEmail(String email)`
      - `Optional<User> findByUsername(String username)`
      - `boolean existsByEmail(String email)`
      - `boolean existsByEmailAndIdNot(String email, Long id)`
      - `boolean existsByEmailIgnoreCase(String email)`
      - `boolean existsByUsername(String username)`
      - `boolean existsByUsernameAndIdNot(String username, Long id)`
      - `boolean existsByUsernameIgnoreCase(String username)`

`backend/src/main/java/com/smartcampus/maintenance/security/AuthenticatedUser.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.security`
* Types and Methods:
  - `class AuthenticatedUser`
    - Constructors:
      - `public AuthenticatedUser(User user)`
    - Methods:
      - `public Collection<GrantedAuthority> getAuthorities()`
      - `public Long getId()`
      - `public Role getRole()`

    - `public String getPassword()`
    - `public String getUsername()`
    - `public boolean isAccountNonExpired()`
    - `public boolean isAccountNonLocked()`
    - `public boolean isCredentialsNonExpired()`
    - `public boolean isEnabled()`
    - `public int getTokenVersion()`

`backend/src/main/java/com/smartcampus/maintenance/security/CustomUserDetailsService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.security`
* Types and Methods:
  - `class CustomUserDetailsService`
    - Constructors:
      - `public CustomUserDetailsService(UserRepository userRepository)`
    - Methods:
      - `public UserDetails loadUserByUsername(String username)`

`backend/src/main/java/com/smartcampus/maintenance/security/JwtAuthenticationFilter.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.security`
* Types and Methods:
  - `class JwtAuthenticationFilter`
    - Constructors:
      - `public JwtAuthenticationFilter(JwtService jwtService, CustomUserDetailsService userDetailsService)`
    - Methods:
      - `protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)`

`backend/src/main/java/com/smartcampus/maintenance/security/JwtService.java`

* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class JwtService`
    - Methods:
      - `Claims parseClaims(String token)`
      - `Instant resolveExpirationInstant()`
      - `String extractUsername(String token)`
      - `String generateToken(User user)`
      - `boolean isTokenValid(String token, UserDetails userDetails)`
      - `long getExpirationMs()`
      - `throw new IllegalStateException("JWT secret must be at least 32 bytes")`
      - `void init()`

`backend/src/main/java/com/smartcampus/maintenance/service/AnalyticsService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class AnalyticsService`
    - Constructors:
      - `public AnalyticsService(TicketRepository ticketRepository)`
    - Methods:

- `private String resolveBuildingName(Ticket ticket)`
- `private double averageHours(List<Ticket> tickets)`
- `private double round(double value)`
- `private void requireAdmin(User actor)`
- `public AnalyticsSummaryResponse getSummary(User actor)`
- `public List<CrewPerformanceResponse> getCrewPerformance(User actor)`
- `public List<TopBuildingResponse> getTopBuildings(User actor)`
- `public PublicLandingStatsResponse getPublicLandingStats()`
- `public ResolutionTimeResponse getResolutionTime(User actor)`

`backend/src/main/java/com/smartcampus/maintenance/service/AnnouncementService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class AnnouncementService`
    - Constructors:
      - `public AnnouncementService(AnnouncementRepository announcementRepository, UserRepository userRepository,
NotificationDispatchService notificationDispatchService)`
    - Methods:
      - `private AnnouncementResponse toResponse(Announcement a)`
      - `private void requireAdmin(User actor)`
      - `public AnnouncementResponse create(User actor, AnnouncementCreateRequest request)`
      - `public List<AnnouncementResponse> getActiveAnnouncements()`
      - `public List<AnnouncementResponse> getAllAnnouncements(User actor)`
      - `public void toggleActive(Long id, User actor)`

`backend/src/main/java/com/smartcampus/maintenance/service/AuditEventService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class AuditEventService`
    - Constructors:
      - `public AuditEventService(AuditEventRepository auditEventRepository, ObjectMapper objectMapper)`
    - Methods:
      - `private String serializeDetails(Object details)`
      - `public void record(String action, User actor, String targetType, String targetId, RequestMetadata metadata,
Object details)`

`backend/src/main/java/com/smartcampus/maintenance/service/AuthRefreshTokenService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class AuthRefreshTokenService`
    - Constructors:
      - `public AuthRefreshTokenService(AuthRefreshTokenRepository authRefreshTokenRepository, TokenHashService tokenHashService, long refreshTokenTtlDays)`
    - Methods:
      - `private AuthRefreshToken loadToken(String rawToken)`
      - `private void applyMetadata(AuthRefreshToken token, RequestMetadata metadata)`
      - `private void revokeFamily(String tokenFamily)`
      - `public IssuedRefreshToken issue(User user, RequestMetadata metadata)`
      - `public IssuedRefreshToken rotate(String rawToken, User user, RequestMetadata metadata)`

- `public User consumeForRefresh(String rawToken, RequestMetadata metadata)`
- `public long cleanupExpiredOrRevoked(LocalDateTime expiredBefore, LocalDateTime revokedBefore)`
- `public record IssuedRefreshToken(String rawToken, LocalDateTime expiresAt)`
- `public void revoke(String rawToken)`
- `public void revokeAllForUser(Long userId)`

`backend/src/main/java/com/smartcampus/maintenance/service/AuthService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class AuthService`
    - Constructors:
      - `public AuthService(AuthenticationManager authenticationManager, UserRepository userRepository, PendingRegistrationRepository pendingRegistrationRepository, PasswordEncoder passwordEncoder, JwtService jwtService, PasswordResetTokenRepository resetTokenRepository, AuthMfaChallengeRepository mfaChallengeRepository, StaffInviteRepository staffInviteRepository, UserService userService, PasswordPolicyService passwordPolicyService, TokenHashService tokenHashService, EmailService emailService, AuthRefreshTokenService authRefreshTokenService, RefreshCookieService refreshCookieService, AuditEventService auditEventService, ApplicationEventPublisher eventPublisher, PlatformTransactionManager transactionManager, String frontendBaseUrl, long verificationCodeTtlMinutes, long resetTokenTtlMinutes, long verificationResendCooldownSeconds, long resetRequestCooldownSeconds, long mfaCodeTtlMinutes, long mfaResendCooldownSeconds, int mfaCodeMaxAttempts, List<String> mfaRequiredRoles, long publicRequestMinDelayMs)`
    - Methods:
      - `private AuthResponse buildAuthResponse(User user)`
      - `private Duration durationUntil(LocalDateTime expiresAt)`
      - `private IssuedMfaChallenge issueMfaChallenge(User user)`
      - `private String buildLoginUrl()`
      - `private String buildResetUrl(String token)`
      - `private String generateNumericCode(int length)`
      - `private String generateUniqueResetToken()`
      - `private String rotateVerificationCode(PendingRegistration pending)`
      - `private boolean isCooldownActive(LocalDateTime createdAt, long cooldownSeconds)`
      - `private boolean isResendCooldownActive(PendingRegistration pending)`
      - `private boolean requiresMfa(User user)`
      - `private record IssuedMfaChallenge(String challengeId, String rawCode)`
      - `private void clearVerificationCode(PendingRegistration pending)`
      - `private void clearVerificationCodeInNewTransaction(Long pendingRegistrationId)`
      - `private void enforceMinimumPublicDelay(long startedAtNs)`
      - `private void ensureUsernameAvailable(String username, String fullName, Long pendingRegistrationId)`
      - `public AuthResponse login(LoginRequest request, RequestMetadata metadata, HttpHeaders responseHeaders)`
      - `public AuthResponse refreshSession(String rawRefreshToken, RequestMetadata metadata, HttpHeaders responseHeaders)`
      - `public AuthResponse verifyMfa(String challengeId, String code, RequestMetadata metadata, HttpHeaders responseHeaders)`
      - `public CurrentUserResponse currentUser(User user)`
      - `public List<String> getUsernameSuggestions(String preferredUsername, String fullName)`
      - `public void acceptStaffInvite(AcceptStaffInviteRequest request, RequestMetadata metadata)`
      - `public void forgotPassword(String email, RequestMetadata metadata)`
      - `public void logout(String rawRefreshToken, RequestMetadata metadata, HttpHeaders responseHeaders)`
      - `public void registerStudent(RegisterRequest request, RequestMetadata metadata)`

- `public void resendMfaCode(String challengeId, RequestMetadata metadata)`
- `public void resendVerificationCode(String email, RequestMetadata metadata)`
- `public void resetPassword(String token, String newPassword, RequestMetadata metadata)`
- `public void verifyEmail(String email, String code, RequestMetadata metadata)`

`backend/src/main/java/com/smartcampus/maintenance/service/AuthTokenCleanupScheduler.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class AuthTokenCleanupScheduler`
    - Constructors:
      - `public AuthTokenCleanupScheduler(PasswordResetTokenRepository resetTokenRepository, EmailVerificationTokenRepository verificationTokenRepository, AuthMfaChallengeRepository mfaChallengeRepository, AuthRefreshTokenService authRefreshTokenService, long usedTokenRetentionHours)`
    - Methods:
      - `public void cleanupAuthTokens()`

`backend/src/main/java/com/smartcampus/maintenance/service/AutoAssignmentService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class AutoAssignmentService`
    - Constructors:
      - `public AutoAssignmentService(UserRepository userRepository, TicketRepository ticketRepository)`
    - Methods:
      - `private AssignmentCandidateMetrics buildCandidateMetrics(Ticket ticket, User user)`
      - `private List<String> buildReasons(AssignmentCandidateMetrics candidate, int lowestActiveWorkload)`
      - `private TicketAssignmentRecommendationResponse toRecommendation(AssignmentCandidateMetrics candidate, double score, int lowestActiveWorkload)`
      - `public List<TicketAssignmentRecommendationResponse> recommendAssignees(Ticket ticket, int limit)`
      - `public Optional<User> findBestAssignee(Ticket ticket)`
      - `public Optional<User> findBestAssigneeWithinCapacity(Ticket ticket, int maxActiveOpenTickets)`

`backend/src/main/java/com/smartcampus/maintenance/service/BuildingService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class BuildingService`
    - Constructors:
      - `public BuildingService(BuildingRepository buildingRepository, CatalogEventStreamService catalogEventStreamService, AuditEventService auditEventService)`
    - Methods:
      - `private BuildingResponse toResponse(Building b)`
      - `private int resolveSortOrder(Integer requestedSortOrder)`
      - `private void requireAdmin(User actor)`
      - `private void requireOperationalUser(User actor)`
      - `public Building requireActiveBuilding(Long buildingId)`
      - `public BuildingResponse createBuilding(User actor, BuildingCreateRequest request)`
      - `public BuildingResponse updateBuilding(User actor, Long buildingId, BuildingUpdateRequest request)`
      - `public List<BuildingResponse> getActiveBuildings()`
      - `public List<BuildingResponse> getAllBuildings(User actor)`
      - `public List<BuildingResponse> getOperationalBuildings(User actor)`

`backend/src/main/java/com/smartcampus/maintenance/service/CaptchaVerificationService.java`
* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class CaptchaVerificationService`
    - Constructors:
      - `public CaptchaVerificationService(ObjectMapper objectMapper, boolean enabled, String secretKey, String verifyUrl)`
    - Methods:
      - `private String encode(String value)`
      - `public void verify(String captchaToken, String remoteIp)`

`backend/src/main/java/com/smartcampus/maintenance/service/CatalogEventStreamService.java`
* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class CatalogEventStreamService`
    - Methods:
      - `private void send(SseEmitter emitter, CatalogStreamEvent event)`
      - `public SseEmitter subscribe()`
      - `public void publish(String resource, String action, List<Long> changedIds)`

`backend/src/main/java/com/smartcampus/maintenance/service/CatalogService.java`
* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class CatalogService`
    - Constructors:
      - `public CatalogService(ServiceDomainRepository serviceDomainRepository, RequestTypeRepository requestTypeRepository, SupportCategoryRepository supportCategoryRepository, CatalogEventStreamService catalogEventStreamService, AuditEventService auditEventService)`
    - Methods:
      - `private RequestTypeResponse toRequestTypeResponse(RequestType requestType)`
      - `private ServiceDomainResponse toServiceDomainResponse(ServiceDomain serviceDomain)`
      - `private SupportCategoryResponse toSupportCategoryResponse(SupportCategory supportCategory)`
      - `private int resolveRequestTypeSortOrder(Integer requestedSortOrder, String serviceDomainKey)`
      - `private int resolveSupportCategorySortOrder(Integer requestedSortOrder)`
      - `private void requireAdmin(User actor)`
      - `public List<RequestTypeResponse> getActiveRequestTypes(String serviceDomainKey)`
      - `public List<RequestTypeResponse> getAllRequestTypes(User actor)`
      - `public List<ServiceDomainResponse> getServiceDomains()`
      - `public List<SupportCategoryResponse> getActiveSupportCategories()`
      - `public List<SupportCategoryResponse> getAllSupportCategories(User actor)`
      - `public RequestType requireActiveRequestType(Long requestTypeId)`
      - `public RequestTypeResponse createRequestType(User actor, RequestTypeCreateRequest request)`
      - `public RequestTypeResponse updateRequestType(User actor, Long requestTypeId, RequestTypeUpdateRequest request)`
      - `public SupportCategory requireActiveSupportCategory(Long supportCategoryId)`
      - `public SupportCategoryResponse createSupportCategory(User actor, SupportCategoryCreateRequest request)`
      - `public SupportCategoryResponse updateSupportCategory(User actor, Long supportCategoryId, SupportCategoryUpdateRequest request)`

`backend/src/main/java/com/smartcampus/maintenance/service/ChatService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class ChatService`
    - Constructors:
      - `public ChatService(ChatMessageRepository chatMessageRepository, TicketRepository ticketRepository, UserRepository userRepository, NotificationDispatchService notificationDispatchService)`
    - Methods:
      - `private ChatMessageResponse toResponse(ChatMessage m)`
      - `private Ticket requireTicket(Long ticketId)`
      - `private void ensureAccess(Ticket ticket, User actor)`
      - `private void notifyChatParticipants(Ticket ticket, User actor, String content)`
      - `public ChatMessageResponse sendMessage(Long ticketId, ChatSendRequest request, User actor)`
      - `public List<ChatMessageResponse> getMessages(Long ticketId, User actor)`

`backend/src/main/java/com/smartcampus/maintenance/service/CurrentUserService.java`
* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class CurrentUserService`
    - Methods:
      - `User requireCurrentUser()`
      - `public CurrentUserService(UserRepository userRepository)`
      - `throw new UnauthorizedException("Authentication required")`

`backend/src/main/java/com/smartcampus/maintenance/service/EmailDeliveryService.java`
* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class EmailDeliveryService`
    - Methods:
      - ` sendHtml(to, subject, plainText, htmlBody)`
      - ` sendPlain(to, subject, plainText)`
      - `RuntimeException enrichDeliveryException(Exception ex)`
      - `boolean canSend(String to, String subject)`
      - `boolean isGmailHost(String host)`
      - `return new IllegalStateException("Failed to deliver email", ex)`
      - `return new IllegalStateException(message, ex)`
      - `throw enrichDeliveryException(ex)`
      - `void send(String to, String subject, String plainText, String htmlBody)`
      - `void sendHtml(String to, String subject, String plainText, String htmlBody)`
      - `void sendPlain(String to, String subject, String body)`

`backend/src/main/java/com/smartcampus/maintenance/service/EmailOutboxScheduler.java`
* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class EmailOutboxScheduler`
    - Constructors:
      - `public EmailOutboxScheduler(EmailOutboxService emailOutboxService)`
    - Methods:
      - `public void cleanupSentEmails()`

- `public void processPendingEmails()`

`backend/src/main/java/com/smartcampus/maintenance/service/EmailOutboxService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class EmailOutboxService`
    - Constructors:
      - `public EmailOutboxService(EmailOutboxRepository emailOutboxRepository, EmailDeliveryService emailDeliveryService, boolean emailEnabled, int batchSize, int maxAttempts, long retentionDays)`
    - Methods:
      - `private String safeError(String message)`
      - `private void deliver(EmailOutbox message)`
      - `public int processPendingBatch()`
      - `public long cleanupSentHistory()`
      - `public void enqueue(String toEmail, String subject, String plainText, String htmlBody)`

`backend/src/main/java/com/smartcampus/maintenance/service/EmailService.java`

* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class EmailService`
    - Methods:
      - ` sendBrandedEmail(toEmail, "CampusFix: New Assignment", text, "Work assigned", "A new work order is waiting for you.", "Open CampusFix to review the location, urgency, and attached request details.", "#" + ticketId, "Open Work Queue", frontendBaseUrl, "This assignment was generated by CampusFix operations.")`
      - ` sendBrandedEmail(toEmail, "CampusFix: Password Changed", text, "Security notice", "Your password was changed successfully.", "If this was not you, contact support immediately and sign in to review your account activity.", null, "Sign In", loginUrl, "For security, keep your password unique and never share it.")`
      - ` sendBrandedEmail(toEmail, "CampusFix: Password Reset Request", text, "Password reset", "Reset your CampusFix password.", "Use the secure link below to choose a new password for your account.", null, "Reset Password", resetUrl, "This link expires in " + expiresInMinutes + " minutes. If you did not request this, you can ignore this email.")`
      - ` sendBrandedEmail(toEmail, "CampusFix: SLA Breach Alert", text, "SLA alert", "A ticket has breached its response target.", "Open CampusFix immediately to review the delay, assign the next action, and recover service time.", "#" + ticketId, "Open CampusFix", frontendBaseUrl, "This alert was sent because the ticket exceeded its SLA deadline.")`
      - ` sendBrandedEmail(toEmail, "CampusFix: Ticket Created", text, "Request received", "Your maintenance request has been submitted.", "CampusFix recorded your request and the operations team can now review it.", "#" + ticketId, "Open CampusFix", frontendBaseUrl, "Keep this ticket ID for future reference: #" + ticketId + ".")`
      - ` sendBrandedEmail(toEmail, "CampusFix: Ticket Resolved", text, "Request resolved", "Your maintenance request has been marked resolved.", "Open CampusFix to review the result, follow-up notes, and rating prompt.", "#" + ticketId, "Review Ticket", frontendBaseUrl, "If the issue is not fully resolved, reply through CampusFix support.")`
      - ` sendBrandedEmail(toEmail, "CampusFix: Verify Your Email", text, "Verify account", "Finish creating your CampusFix account.", "Enter the verification code below in CampusFix to verify your email and finish creating your account.", verificationCode, null, null, "This verification code expires in " + expiresInMinutes + " minutes.")`
      - ` sendBrandedEmail(toEmail, "CampusFix: Your Sign-in Code", text, "Security check", "Confirm your sign-in", "Enter this one-time code in CampusFix to complete your sign-in.", verificationCode, null, null, "This code expires in " + expiresInMinutes + " minutes. If this wasn't you, reset your password.")`
      - ` sendBrandedEmail(toEmail, "Welcome to CampusFix", text, "Account ready", "Your CampusFix account is now

active.", "You can start reporting and tracking campus maintenance issues from one secure workspace.", null, "Sign In", loginUrl, "If you did not expect this email, contact CampusFix support immediately.")`
  - ` sendPlainEmail(supportInbox, "CampusFix Support Request: " + subject, body)`
  - `String displayName(String fullName)`
  - `void sendBrandedEmail(String to, String subject, String plainText, String eyebrow, String heading, String intro, String highlightValue, String buttonLabel, String buttonUrl, String note)`
  - `void sendMfaCodeEmail(String fullName, String toEmail, String verificationCode, long expiresInMinutes)`
  - `void sendPasswordChangedEmail(String fullName, String toEmail, String loginUrl)`
  - `void sendPasswordResetEmail(String fullName, String toEmail, String resetUrl, long expiresInMinutes)`
  - `void sendPlainEmail(String to, String subject, String body)`
  - `void sendSlaBreachEmail(String toEmail, String ticketTitle, Long ticketId)`
  - `void sendStaffInviteEmail(String fullName, String toEmail, String acceptUrl, long expiresInHours)`
  - `void sendSupportRequestEmail(String fullName, String fromEmail, String category, String subject, String message)`
  - `void sendTicketAssignedEmail(String toEmail, String ticketTitle, Long ticketId)`
  - `void sendTicketCreatedEmail(String toEmail, String ticketTitle, Long ticketId)`
  - `void sendTicketResolvedEmail(String toEmail, String ticketTitle, Long ticketId)`
  - `void sendVerificationCodeEmail(String fullName, String toEmail, String verificationCode, long expiresInMinutes)`
  - `void sendWelcomeEmail(String fullName, String toEmail, String loginUrl)`

`backend/src/main/java/com/smartcampus/maintenance/service/EscalationScheduler.java`

* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class EscalationScheduler`
    - Methods:
      - `UrgencyLevel bumpUrgency(UrgencyLevel current)`
      - `public EscalationScheduler(TicketRepository ticketRepository, SlaService slaService, NotificationService notificationService, UserRepository userRepository, EmailService emailService)`
      - `void escalateBreachedTickets()`

`backend/src/main/java/com/smartcampus/maintenance/service/NotificationDispatchService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class NotificationDispatchService`
    - Constructors:
      - `public NotificationDispatchService(NotificationService notificationService)`
    - Methods:
      - `public void notifyUser(User user, String title, String message, NotificationType type, String linkUrl)`
      - `public void notifyUsers(Collection<User> users, String title, String message, NotificationType type, String linkUrl)`

`backend/src/main/java/com/smartcampus/maintenance/service/NotificationService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class NotificationService`
    - Constructors:
      - `public NotificationService(NotificationRepository notificationRepository)`
    - Methods:
      - `private NotificationResponse toResponse(Notification n)`

- `public List<NotificationResponse> getNotifications(User actor)`
- `public long getUnreadCount(User actor)`
- `public void markAllRead(User actor)`
- `public void markRead(Long notificationId, User actor)`
- `public void notify(User user, String title, String message, NotificationType type, String linkUrl)`

`backend/src/main/java/com/smartcampus/maintenance/service/PasswordPolicyService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class PasswordPolicyService`
    - Methods:
      - `private void addToken(Set<String> tokens, String value)`
      - `public ValidationResult evaluate(String password, String username, String email, String fullName)`
      - `public record ValidationResult(boolean valid, String message)`
      - `public void enforce(String password, String username, String email, String fullName)`

`backend/src/main/java/com/smartcampus/maintenance/service/PendingRegistrationEmailListener.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class PendingRegistrationEmailListener`
    - Constructors:
      - `public PendingRegistrationEmailListener(EmailService emailService)`
    - Methods:
      - `public void onPendingRegistrationVerificationRequested(PendingRegistrationVerificationRequestedEvent event)`
      - `public void onPendingRegistrationVerified(PendingRegistrationVerifiedEvent event)`

`backend/src/main/java/com/smartcampus/maintenance/service/PublicEndpointSecurityService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class PublicEndpointSecurityService`
    - Constructors:
      - `public PublicEndpointSecurityService(RateLimitService rateLimitService, CaptchaVerificationService captchaVerificationService, long windowSeconds, int loginIpLimit, int loginAccountLimit, int registerIpLimit, int forgotPasswordIpLimit, int resendVerificationIpLimit, int supportIpLimit)`
    - Methods:
      - `private void enforceCaptchaAndRateLimit(String scope, String email, String captchaToken, RequestMetadata metadata, int limit)`
      - `public void guardForgotPassword(String email, String captchaToken, RequestMetadata metadata)`
      - `public void guardLogin(String username, String captchaToken, RequestMetadata metadata)`
      - `public void guardRegistration(String email, String captchaToken, RequestMetadata metadata)`
      - `public void guardResendVerification(String email, String captchaToken, RequestMetadata metadata)`
      - `public void guardSupport(String email, String captchaToken, RequestMetadata metadata)`

`backend/src/main/java/com/smartcampus/maintenance/service/PublicLandingConfigService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class PublicLandingConfigService`

  - Methods:
    - `public PublicLandingConfigResponse getPublicConfig()`

`backend/src/main/java/com/smartcampus/maintenance/service/RateLimitService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class RateLimitService`
    - Constructors:
      - `public RateLimitService(ObjectProvider<StringRedisTemplate> redisTemplateProvider, boolean enabled, boolean redisEnabled)`
    - Methods:
      - `private record Counter(long windowStartMs, int count)`
      - `public void enforce(String scope, String key, int limit, Duration window)`

`backend/src/main/java/com/smartcampus/maintenance/service/RefreshCookieService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class RefreshCookieService`
    - Constructors:
      - `public RefreshCookieService(String cookieName, String sameSite, boolean secure, String domain)`
    - Methods:
      - `public String cookieName()`
      - `public void clearRefreshCookie(HttpHeaders headers)`
      - `public void writeRefreshCookie(HttpHeaders headers, String token, Duration maxAge)`

`backend/src/main/java/com/smartcampus/maintenance/service/ReportService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class ReportService`
    - Constructors:
      - `public ReportService(TicketRepository ticketRepository)`
    - Methods:
      - `private String escape(String value)`
      - `private String resolveBuildingName(Ticket ticket)`
      - `private void requireAdmin(User actor)`
      - `public byte[] exportTicketsCsv(User actor)`

`backend/src/main/java/com/smartcampus/maintenance/service/RequestMetadata.java`

* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type unknown`
    - Methods:
      - `record RequestMetadata(String ipAddress, String userAgent)`

`backend/src/main/java/com/smartcampus/maintenance/service/RequestMetadataResolver.java`

* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:

  - `type class RequestMetadataResolver`
    - Methods:
      - ` CidrMatcher(InetAddress address, int prefixLength)`
      - ` ExactAddressMatcher(InetAddress address)`
      - `RequestMetadata resolve(HttpServletRequest request)`
      - `String firstForwardedIp(String value)`
      - `String normalizeIp(String value)`
      - `String sanitize(String value, int maxLength)`
      - `String stripPortAndBrackets(String value)`
      - `TrustedProxyMatcher parseMatcher(String value)`
      - `boolean isTrustedProxy(String remoteAddress)`
      - `boolean matches(InetAddress address)`
      - `return new CidrMatcher(network, prefixLength)`
      - `return new RequestMetadata(null, null)`
      - `throw new IllegalStateException("Invalid trusted proxy entry: " + value, ex)`
  - `type interface TrustedProxyMatcher`
    - Methods:
      - ` CidrMatcher(InetAddress address, int prefixLength)`
      - ` ExactAddressMatcher(InetAddress address)`
      - `RequestMetadata resolve(HttpServletRequest request)`
      - `String firstForwardedIp(String value)`
      - `String normalizeIp(String value)`
      - `String sanitize(String value, int maxLength)`
      - `String stripPortAndBrackets(String value)`
      - `TrustedProxyMatcher parseMatcher(String value)`
      - `boolean isTrustedProxy(String remoteAddress)`
      - `boolean matches(InetAddress address)`
      - `return new CidrMatcher(network, prefixLength)`
      - `return new RequestMetadata(null, null)`
      - `throw new IllegalStateException("Invalid trusted proxy entry: " + value, ex)`

`backend/src/main/java/com/smartcampus/maintenance/service/ScheduledBroadcastDispatcher.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class ScheduledBroadcastDispatcher`
    - Constructors:
      - `public ScheduledBroadcastDispatcher(ScheduledBroadcastService scheduledBroadcastService)`
    - Methods:
      - `public void dispatchDueScheduledBroadcasts()`

`backend/src/main/java/com/smartcampus/maintenance/service/ScheduledBroadcastService.java`

* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class ScheduledBroadcastService`
    - Methods:
      - ` requireAdmin(actor)`
      - `List<ScheduledBroadcastResponse> list(User actor)`
      - `List<User> resolveRecipients(BroadcastAudience audience, User actor)`
      - `ScheduledBroadcastResponse cancel(User actor, Long scheduledId)`
      - `ScheduledBroadcastResponse schedule(User actor, ScheduledBroadcastCreateRequest request)`
      - `ScheduledBroadcastResponse toResponse(ScheduledBroadcast scheduled)`

- `int dispatchDueScheduledBroadcasts()`
- `public ScheduledBroadcastService(ScheduledBroadcastRepository scheduledBroadcastRepository, UserRepository
userRepository, NotificationDispatchService notificationDispatchService)`
- `throw new BadRequestException("Only pending scheduled broadcasts can be cancelled.")`
- `throw new BadRequestException("Scheduled date/time must be at least 1 minute in the future.")`
- `throw new ForbiddenException("ADMIN role is required")`
- `void requireAdmin(User actor)`

`backend/src/main/java/com/smartcampus/maintenance/service/SlaService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class SlaService`
    - Constructors:
      - `public SlaService(TicketRepository ticketRepository)`
    - Methods:
      - `private boolean isOnTime(Ticket ticket)`
      - `private void requireAdmin(User actor)`
      - `public SlaComplianceResponse getSlaCompliance(User actor)`
      - `public boolean isSlaBreached(Ticket ticket)`

`backend/src/main/java/com/smartcampus/maintenance/service/StaffInviteCleanupScheduler.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class StaffInviteCleanupScheduler`
    - Constructors:
      - `public StaffInviteCleanupScheduler(StaffInviteRepository staffInviteRepository, long usedTokenRetentionHours)`
    - Methods:
      - `public void cleanupInvites()`

`backend/src/main/java/com/smartcampus/maintenance/service/SupportRequestService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class SupportRequestService`
    - Constructors:
      - `public SupportRequestService(SupportRequestRepository supportRequestRepository, CatalogService catalogService,
EmailService emailService, AuditEventService auditEventService)`
    - Methods:
      - `public SupportContactResponse submit(SupportContactRequest request, RequestMetadata metadata)`

**`backend/src/main/java/com/smartcampus/maintenance/service/TicketAttachmentAccessService.java
`**

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class TicketAttachmentAccessService`
    - Constructors:
      - `public TicketAttachmentAccessService(FileStorageService fileStorageService, String signingSecret, long

ttlSeconds)`
  - Methods:
    - `private String sign(Long ticketId, AttachmentType type, String canonicalReference, long expiresAt)`
    - `public String buildSignedUrl(Ticket ticket, AttachmentType type, String storedPath)`
    - `public void validate(Ticket ticket, AttachmentType type, String storedPath, Long expiresAt, String signature)`
- `enum TicketAttachmentAccessService.AttachmentType`
  - Methods:
    - `public String pathSegment()`
    - `public static AttachmentType fromPathSegment(String value)`

`backend/src/main/java/com/smartcampus/maintenance/service/TicketService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
- `class TicketService`
  - Constructors:
    - `public TicketService(TicketRepository ticketRepository, TicketLogRepository ticketLogRepository, TicketRatingRepository ticketRatingRepository, TicketCommentRepository ticketCommentRepository, UserRepository userRepository, BuildingService buildingService, CatalogService catalogService, AutoAssignmentService autoAssignmentService, FileStorageService fileStorageService, TicketAttachmentAccessService ticketAttachmentAccessService, NotificationDispatchService notificationDispatchService, EmailService emailService)`
  - Methods:
    - `private CommentResponse toCommentResponse(TicketComment c)`
    - `private String resolveBuildingName(Ticket ticket)`
    - `private String safeNote(String note, String fallback)`
    - `private String ticketLink(Ticket ticket)`
    - `private Ticket requireTicket(Long ticketId)`
    - `private TicketResponse toResponse(Ticket ticket)`
    - `private boolean tryAutoAssign(Ticket ticket, User actor)`
    - `private double similarity(String a, String b)`
    - `private int levenshtein(String a, String b)`
    - `private void addLog(Ticket ticket, TicketStatus oldStatus, TicketStatus newStatus, User changedBy, String note)`
    - `private void ensureAccess(Ticket ticket, User actor)`
    - `private void ensureCanUpdateStatus(Ticket ticket, TicketStatusUpdateRequest request, User actor)`
    - `private void notifyAdmins(String title, String message, NotificationType type, String linkUrl)`
    - `private void notifyTicketStakeholders(Ticket ticket, User actor, String title, String message)`
    - `private void notifyTicketStakeholders(Ticket ticket, User actor, String title, String message, NotificationType type)`
    - `private void requireRole(User actor, Role required)`
    - `public CommentResponse addComment(Long ticketId, CommentCreateRequest request, User actor)`
    - `public DuplicateCheckResponse checkDuplicates(TicketCreateRequest request)`
    - `public List<CommentResponse> getComments(Long ticketId, User actor)`
    - `public List<TicketAssignmentRecommendationResponse> getAssignmentRecommendations(Long ticketId, User actor)`
    - `public List<TicketLogResponse> getLogs(Long ticketId, User actor)`
    - `public List<TicketResponse> getAllTickets(User actor, TicketStatus status, String serviceDomainKey, Long requestTypeId, Long buildingId, UrgencyLevel urgency, Long assigneeId, String search)`
    - `public List<TicketResponse> getAssignedTickets(User actor)`
    - `public List<TicketResponse> getMyTickets(User actor)`
    - `public ResponseEntity<Resource> downloadAttachment(Long ticketId, String attachmentType, Long expiresAt, String signature)`
    - `public TicketDetailResponse getTicketDetail(Long ticketId, User actor)`

- `public TicketRatingResponse rateTicket(Long ticketId, TicketRateRequest request, User actor)`
- `public TicketResponse assignTicket(Long ticketId, TicketAssignRequest request, User actor)`
- `public TicketResponse createTicket(User actor, TicketCreateRequest request, MultipartFile imageFile)`
- `public TicketResponse respondToAssignment(Long ticketId, TicketAssignmentResponseRequest request, User actor)`
- `public TicketResponse updateStatus(Long ticketId, TicketStatusUpdateRequest request, User actor)`
- `public TicketResponse uploadAfterPhoto(Long ticketId, MultipartFile image, User actor)`

`backend/src/main/java/com/smartcampus/maintenance/service/TokenHashService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class TokenHashService`
    - Methods:
      - `public String generateUrlToken(int sizeBytes)`
      - `public String hashSha256(String rawValue)`

`backend/src/main/java/com/smartcampus/maintenance/service/UserService.java`

* Language: `java`
* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class UserService`
    - Methods:
      - ` requireAdmin(actor)`
      - `BroadcastMessageResponse broadcastMessage(User actor, BroadcastMessageRequest request)`
      - `List<String> suggestAvailableUsernames(String preferredUsername, String fullName, int limit)`
      - `List<UserSummaryResponse> getMaintenanceUsers(User actor)`
      - `List<UserWithTicketCountResponse> getAllUsersWithTicketCount(User actor)`
      - `StaffInviteResponse inviteStaffUser(User actor, StaffInviteRequest request)`
      - `String buildInviteUrl(String token)`
      - `UserProfileResponse getMyProfile(User actor)`
      - `UserProfileResponse toProfile(User user)`
      - `UserProfileResponse updateMyProfile(User actor, UserProfileUpdateRequest request)`
      - `boolean isEmailUnavailable(String email)`
      - `boolean isUsernameUnavailable(String username)`
      - `long resolveTicketCount(User user)`
      - `return toProfile(actor)`
      - `return toProfile(saved)`
      - `throw new ConflictException("Email is already registered or has a pending invitation.")`
      - `throw new ForbiddenException("ADMIN role is required")`
      - `void requireAdmin(User actor)`

`backend/src/main/java/com/smartcampus/maintenance/service/UsernameSuggestionService.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class UsernameSuggestionService`
    - Methods:
      - `private String sanitizeBase(String preferredUsername, String fullName)`
      - `public List<String> generate(String preferredUsername, String fullName, int limit)`

`backend/src/main/java/com/smartcampus/maintenance/util/FileStorageService.java`

* Language: `java`

* Purpose: Business/service logic
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class FileStorageService`
    - Methods:
      - ` this(uploadDir, allowedContentTypes, maxFileSizeBytes, 5, 85, 85)`
      - ` validateImage(file)`
      - `StoredFile load(String storedPath)`
      - `String canonicalStoredReference(String storedPath)`
      - `String detectContentType(Path path, String filename)`
      - `String normalizeContentType(String contentType)`
      - `String resolveExtension(String contentType)`
      - `String store(MultipartFile file)`
      - `boolean hasExpectedFileSignature(MultipartFile file, String contentType)`
      - `boolean hasExpectedFileSignature(byte[] bytes, String contentType)`
      - `boolean isOptimizationUsable(OptimizedImageResult result, String expectedContentType, int originalSize)`
      - `byte[] maybeOptimizeImage(byte[] originalBytes, String contentType)`
      - `byte[] readBytes(MultipartFile file)`
      - `int clampQuality(int quality)`
      - `public FileStorageService(String uploadDir, List<String> allowedContentTypes, long maxFileSizeBytes)`
      - `record StoredFile(Path path, String filename, String contentType)`
      - `return hasExpectedFileSignature(header, contentType)`
      - `throw new BadRequestException("Failed to inspect uploaded file.")`
      - `throw new BadRequestException("Failed to read uploaded file.")`
      - `throw new BadRequestException("Failed to store file")`
      - `throw new BadRequestException("File exceeds the maximum allowed size.")`
      - `throw new BadRequestException("Unsupported file type. Only JPG, PNG, and WEBP images are allowed.")`
      - `throw new BadRequestException("Uploaded file content does not match the declared image type.")`
      - `throw new IllegalStateException("Could not initialize upload directory", ex)`
      - `throw new NotFoundException("Attachment not found")`
      - `void validateImage(MultipartFile file)`

`backend/src/main/java/com/smartcampus/maintenance/util/ServiceDomainCatalog.java`

* Language: `java`
* Purpose: Project source code
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class ServiceDomainCatalog`
    - Methods:
      - `String defaultRequestTypeLabel(String key)`
      - `String labelForKey(String key)`
      - `TicketCategory legacyCategoryForKey(String key)`
      - `private ServiceDomainCatalog()`

`backend/src/main/resources/db/migration/V1__initial_schema.sql`

* Language: `sql`
* Purpose: Database schema/seed script
* Tables:
  - `announcements`
  - `buildings`
  - `chat_messages`
  - `email_outbox`
  - `email_verification_tokens`
  - `notifications`
  - `password_reset_tokens`

  - `scheduled_broadcasts`
  - `staff_invites`
  - `support_requests`
  - `ticket_comments`
  - `ticket_logs`
  - `ticket_ratings`
  - `tickets`
  - `users`
* Views:
  - None detected.
* Functions/Procedures:
  - None detected.

`backend/src/main/resources/db/migration/V2__service_catalog_normalization.sql`
* Language: `sql`
* Purpose: Database schema/seed script
* Tables:
  - `request_types`
  - `service_domains`
  - `support_categories`
* Views:
  - None detected.
* Functions/Procedures:
  - None detected.

`backend/src/main/resources/db/migration/V3__security_session_hardening.sql`
* Language: `sql`
* Purpose: Database schema/seed script
* Tables:
  - `audit_events`
  - `auth_refresh_tokens`
* Views:
  - None detected.
* Functions/Procedures:
  - None detected.

`backend/src/main/resources/db/migration/V4__expand_email_outbox_body_columns.sql`
* Language: `sql`
* Purpose: Database schema/seed script
* Tables:
  - None detected.
* Views:
  - None detected.
* Functions/Procedures:
  - None detected.

`backend/src/main/resources/db/migration/V5__pending_registrations.sql`
* Language: `sql`
* Purpose: Database schema/seed script
* Tables:
  - `pending_registrations`
* Views:
  - None detected.
* Functions/Procedures:
  - None detected.

`backend/src/main/resources/db/migration/V6__auth_mfa_enablement.sql`

* Language: `sql`
* Purpose: Database schema/seed script
* Tables:
  - `auth_mfa_challenges`
* Views:
  - None detected.
* Functions/Procedures:
  - None detected.

`backend/src/test/java/com/smartcampus/maintenance/ApiFlowIntegrationTest.java`

* Language: `java`
* Purpose: Project source code
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class ApiFlowIntegrationTest`
    - Methods:
      - `String refreshCookieValue(MvcResult result)`
      - `String tokenFor(String username, String password)`
      - `Ticket prepareApprovedTicket()`
      - `Ticket prepareTicketWithImage()`
      - `void adminCanFetchAssignmentRecommendationsForApprovedTicket()`
      - `void attachmentEndpointRejectsUnsignedRequests()`
      - `void attachmentEndpointServesSignedUrls()`
      - `void catalogStreamRequiresAuthentication()`
      - `void healthEndpointIsPublic()`
      - `void loginReturnsSessionPayloadRoleAndRefreshCookie()`
      - `void logoutClearsRefreshCookie()`
      - `void maintenanceCanAccessAssignedTickets()`
      - `void refreshEndpointRotatesRefreshCookie()`
      - `void studentCannotAccessAdminTicketListing()`

`backend/src/test/java/com/smartcampus/maintenance/AuthControllerRegistrationIntegrationTest.java`

* Language: `java`
* Purpose: Project source code
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class AuthControllerRegistrationIntegrationTest`
    - Methods:
      - `void registerCreatesPendingRegistrationInsteadOfUser()`
      - `void registerDuplicateVerifiedEmailReturnsGenericSuccessWithoutCreatingPendingRegistration()`

`backend/src/test/java/com/smartcampus/maintenance/AuthSecurityServiceIntegrationTest.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance`
* Types and Methods:
  - `class AuthSecurityServiceIntegrationTest`
    - Methods:
      - `private RegisterRequest pendingRegistrationRequest()`
      - `private String extractToken(String body)`
      - `private String latestVerificationToken(String email)`
      - `private String sha256(String input)`
      - `private User createUser(boolean emailVerified, String rawPassword)`
      - `void forgotPasswordCooldownPreventsMultipleResetTokens()`
      - `void forgotPasswordUnknownEmailDoesNotCreateResetToken()`

    - `void oldJwtBecomesInvalidWhenTokenVersionChanges()`
    - `void resendVerificationCooldownPreventsImmediateTokenRotation()`
    - `void resetPasswordRejectsSamePasswordAndIncrementsTokenVersion()`
    - `void verifyEmailRejectsExpiredVerificationLink()`

`backend/src/test/java/com/smartcampus/maintenance/PendingRegistrationMigrationTest.java`

* Language: `java`
* Purpose: Project source code
* Parse Notes: Regex fallback parser used.
* Types and Methods:
  - `type class PendingRegistrationMigrationTest`
    - Methods:
      - ` try(Connection connection = DriverManager.getConnection(jdbcUrl, "sa", "")`
      - `void v5MovesUnverifiedUsersIntoPendingRegistrations()`

`backend/src/test/java/com/smartcampus/maintenance/RegistrationEmailOutboxIntegrationTest.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance`
* Types and Methods:
  - `class RegistrationEmailOutboxIntegrationTest`
    - Methods:
      - `private EmailOutbox latestMessageFor(String email)`
      - `private String extractToken(String body)`
      - `void directEmailServiceCallQueuesMessage()`
      - `void registerStudentQueuesVerificationEmailAndStoresPendingRegistration()`
      - `void verifyEmailPromotesPendingRegistrationAndQueuesWelcomeEmail()`

`backend/src/test/java/com/smartcampus/maintenance/SmartCampusMaintenanceApplicationTests.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance`
* Types and Methods:
  - `class SmartCampusMaintenanceApplicationTests`
    - Methods:
      - `void contextLoads()`

`backend/src/test/java/com/smartcampus/maintenance/config/ProductionDeploymentValidatorTest.java`

* Language: `java`
* Purpose: Application/configuration code
* Package: `com.smartcampus.maintenance.config`
* Types and Methods:
  - `class ProductionDeploymentValidatorTest`
    - Methods:
      - `void acceptsValidProductionSettings()`
      - `void rejectsUnsafeProductionDefaults()`
      - `void requiresSmtpSettingsWhenEmailIsEnabled()`

`backend/src/test/java/com/smartcampus/maintenance/service/AutoAssignmentServiceTest.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class AutoAssignmentServiceTest`

  - Methods:
    - `private Ticket sampleTicket()`
    - `private User maintenanceUser(Long id, String username, String fullName)`
    - `private void stubCandidateMetrics(Long userId, long activeOpen, long sameDomain, long sameBuilding, long recent)`
    - `void prefersLowestWorkloadWhenSignalsAreOtherwiseEqual()`
    - `void ranksCandidatesUsingJavaScoring()`

`backend/src/test/java/com/smartcampus/maintenance/service/RequestMetadataResolverTest.java`

* Language: `java`
* Purpose: Business/service logic
* Package: `com.smartcampus.maintenance.service`
* Types and Methods:
  - `class RequestMetadataResolverTest`
    - Methods:
      - `void ignoresSpoofedForwardedHeadersFromUntrustedRemotes()`
      - `void usesForwardedHeadersWhenRequestComesFromTrustedProxy()`

`backend/src/test/java/com/smartcampus/maintenance/util/FileStorageServiceTest.java`

* Language: `java`
* Purpose: Project source code
* Package: `com.smartcampus.maintenance.util`
* Types and Methods:
  - `class FileStorageServiceTest`
    - Methods:
      - `void canonicalStoredReferenceSupportsLegacyUploadUrls()`
      - `void keepsOriginalBytesForWebpWithoutPureJavaEncoder()`
      - `void keepsOriginalBytesWhenJavaOptimizerCannotDecodeImage()`
      - `void rejectsFilesAboveConfiguredSize()`
      - `void rejectsImageWhenSignatureDoesNotMatchContentType()`
      - `void rejectsUnsupportedFileTypes()`
      - `void storesSupportedImagesUsingSafeGeneratedNames()`

`database/schemas/schema.sql`

* Language: `sql`
* Purpose: Database schema/seed script
* Tables:
  - `announcements`
  - `buildings`
  - `chat_messages`
  - `email_outbox`
  - `email_verification_tokens`
  - `notifications`
  - `password_reset_tokens`
  - `pending_registrations`
  - `staff_invites`
  - `support_requests`
  - `ticket_comments`
  - `ticket_logs`
  - `ticket_ratings`
  - `tickets`
  - `users`
* Views:
  - None detected.
* Functions/Procedures:
  - None detected.

`database/seed_data.sql`
* Language: `sql`
* Purpose: Database schema/seed script
* Tables:
  - None detected.
* Views:
  - None detected.
* Functions/Procedures:
  - None detected.

`frontend/eslint.config.js`
* Language: `js`
* Purpose: Project source code
* Exports:
  - `default defineConfig`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/playwright.config.js`
* Language: `js`
* Purpose: Project source code
* Exports:
  - `default defineConfig`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/postcss.config.js`
* Language: `js`
* Purpose: Project source code
* Exports:
  - No named/default exports detected.
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ensureDeclarationSourceFile()`

`frontend/src/App.jsx`
* Language: `jsx`
* Purpose: Project source code
* Exports:
  - `default App`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `App()`
  - `PublicRoute({ children })`
  - `RouteFallback()`
  - `withSuspense(node)`

`frontend/src/components/Admin/AdminConfigurationSection.jsx`
* Language: `jsx`
* Purpose: Reusable UI component

* Exports:
  - `AdminConfigurationSection`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AdminConfigurationSection({ buildings, serviceDomains, requestTypes, supportCategories, onRefresh, })`
  - `EditableBuildingRow({ building, onSave, saving })`
  - `EditableRequestTypeRow({ requestType, onSave, saving })`
  - `EditableSupportCategoryRow({ supportCategory, onSave, saving })`
  - `createBuilding()`
  - `createRequestType()`
  - `createSupportCategory()`
  - `saveBuilding(id, form)`
  - `saveRequestType(id, form)`
  - `saveSupportCategory(id, form)`

`frontend/src/components/Admin/AdminStatCards.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `AdminStatCards`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AdminStatCards({ items = [] })`
  - `StatCard({ label, value, icon, tone = "info", trend, helper, detailRows, detailNote, detailTitle, motionId })`
  - `StatDetail({ item })`
  - `animate(now)`
  - `toMotionId(label, motionId)`
  - `useAnimatedValue(target, duration = 650)`

`frontend/src/components/Admin/AdminToolbar.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `AdminToolbar`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AdminToolbar({ dateRange, onDateRangeChange, onExport, onGenerateReport, className = "", })`

`frontend/src/components/Admin/AnalyticsCharts.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `AnalyticsCharts`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AnalyticsCharts({ tickets = [] })`
  - `ChartCard({ cardId, title, description, detailTitle, detailContent, children })`
  - `aggregateByKey(tickets, key)`
  - `bucketLabel(value, mode)`
  - `renderCategoryChart(heightClass)`
  - `renderResolutionChart(heightClass)`
  - `renderStatusChart(heightClass)`

  - `startOfWeek(value)`

`frontend/src/components/Admin/BroadcastCenter.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `BroadcastCenter`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `BroadcastCenter({ onBroadcast, onSchedule, onCancelScheduled, scheduledEvents = [], scheduledEventsLoading = false,
})`
  - `handleBroadcast(event)`
  - `handleScheduleBroadcast(event)`

`frontend/src/components/Admin/BuildingsRanking.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `BuildingsRanking`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `BuildingsContent({ topBuildings = [], detail = false })`
  - `BuildingsRanking({ topBuildings = [] })`

`frontend/src/components/Admin/CrewPerformance.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `CrewPerformance`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `CrewContent({ crewPerformance = [], resolution, detail = false })`
  - `CrewPerformance({ crewPerformance = [], resolution })`

`frontend/src/components/Admin/ReportBuilder.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `ReportBuilder`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ReportBuilder({ open, onClose, dataSource })`
  - `handleGenerate()`
  - `toggle(key)`

`frontend/src/components/Admin/SLAComplianceCard.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `SLAComplianceCard`
* Classes:
  - No class declarations detected.

* Functions/Components:
  - `SLAComplianceCard({ slaOverview, resolution })`
  - `SLAContent({ slaOverview, resolution, detail = false })`

`frontend/src/components/Admin/StaffOnboarding.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `StaffOnboarding`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `StaffOnboarding({ maintenanceUsers, onInviteStaff, latestInvite, })`
  - `handleSubmit(event)`

`frontend/src/components/Admin/TicketOperationsTable.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `TicketOperationsTable`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `TicketOperationsTable({ tickets, filters, setFilters, maintenanceUsers, onOpenTicket, statuses, serviceDomains, requestTypes, buildings, urgencyLevels, searchValue, onSearchChange, })`
  - `clearFilters()`
  - `getSlaStatus(ticket)`

`frontend/src/components/Admin/UserManagementTable.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `UserManagementTable`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `UserManagementTable({ users = [] })`
  - `roleFilterBar(<select value={roleFilter} onChange={(e)`

`frontend/src/components/Auth/AuthBrandPanel.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `AuthBrandPanel`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AuthBrandPanel({ title = "Welcome to CampusFix", subtitle, icon: Icon, })`

`frontend/src/components/Auth/AuthPasswordField.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `AuthPasswordField`
* Classes:
  - No class declarations detected.
* Functions/Components:

  - `AuthPasswordField({ label, error, registration, autoComplete, placeholder = "Enter password", })`

`frontend/src/components/Auth/AuthShell.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `AuthShell`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AuthShell({ sectionLabel, heading, description, headerAddon, children, footer, aside, // kept for backward-compat ? rendered in single-column fallback documentTitle, taskIcon: TaskIcon, layout = "immersive", heroBrand = false, showHeaderBrand = false, headerBrandSubtitle, brandTitle, brandSubtitle, brandIcon, })`

`frontend/src/components/Auth/OtpCodeField.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `OtpCodeField`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `OtpCodeField({ label = "Verification code", value = "", onChange, onBlur, name, error, inputRef, length = 6, })`
  - `focusInput()`
  - `sanitizeCode(value, length)`
  - `setRefs(node)`

`frontend/src/components/Auth/PasswordChecklist.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `PasswordChecklist`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `PasswordChecklist({ passwordState, show = false, emphasizeInvalid = false, title = "Create a strong password", })`
  - `strengthMeta(level)`

`frontend/src/components/Auth/TurnstileWidget.jsx`

* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `TurnstileWidget`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `TurnstileWidget({ onVerify, className = "" })`
  - `renderWidget()`

`frontend/src/components/Auth/turnstileConfig.js`

* Language: `js`
* Purpose: Reusable UI component
* Exports:
  - `turnstileEnabled`
  - `turnstileSiteKey`
  - `turnstileTestToken`
* Classes:

  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/components/Common/CampusFixLogo.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `CampusFixLogo`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `CampusFixLogo({ collapsed = false, variant = "default", motion = "default", size = "md", subtitle, showWordmark = !collapsed, })`
  - `motionClasses(motion)`

`frontend/src/components/Common/CatalogSyncBridge.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `CatalogSyncBridge`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `CatalogSyncBridge()`
  - `invalidateCatalogResource(queryClient, resource)`

`frontend/src/components/Common/ConfirmDialog.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `ConfirmDialog`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ConfirmDialog({ open, title, message, onConfirm, onCancel, confirmText = "Confirm" })`

`frontend/src/components/Common/DataTable.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `DataTable`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `DataTable({ data = [], columns = [], pageSize = 10, onRowClick, searchable = true, exportable = true, exportFilename = "export", exportTitle = "Report", title, headerActions, emptyTitle = "No data found", emptyMessage = "Try adjusting your filters.", className = "", searchValue, onSearchChange, searchPlaceholder = "Search records...", recordLabel = "records", searchAccessor, })`
  - `SortIcon({ colKey })`
  - `handleExport(format)`
  - `handleSort(key)`
  - `renderCellValue(column, row)`
  - `updateSearch(value)`

`frontend/src/components/Common/DateRangePicker.jsx`
* Language: `jsx`

* Purpose: Reusable UI component
* Exports:
  - `DateRangePicker`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `DateRangePicker({ value, onChange, className = "" })`
  - `applyCustom()`
  - `daysAgo(n)`
  - `formatShort(date)`
  - `handleBlur()`
  - `handleFocus()`
  - `selectPreset(preset)`
  - `startOfMonth()`

`frontend/src/components/Common/EmptyState.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `EmptyState`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `EmptyState({ title, message })`

`frontend/src/components/Common/ExportDropdown.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `ExportDropdown`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ExportDropdown({ onExport, disabled = false, className = "" })`
  - `choose(format)`
  - `handleBlur()`
  - `handleFocus()`

`frontend/src/components/Common/LoadingSpinner.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `LoadingSpinner`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `LoadingSpinner({ label = "Loading..." })`

`frontend/src/components/Common/Modal.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `Modal`
* Classes:
  - No class declarations detected.
* Functions/Components:

  - `Modal({ open, title, onClose, children, width = "max-w-3xl", motionId, panelClassName = "", bodyClassName = "", headerAction = null, transition = DEFAULT_TRANSITION, })`
  - `handleEscape(event)`

`frontend/src/components/Common/ProtectedRoute.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `ProtectedRoute`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ProtectedRoute({ children, roles = [] })`

`frontend/src/components/Common/SkeletonLoader.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `SkeletonLoader`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `SkeletonLoader({ variant = "card", count = 1, className = "" })`

`frontend/src/components/Common/StatusBadge.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `StatusBadge`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `StatusBadge({ status, className })`

`frontend/src/components/Common/UrgencyBadge.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `UrgencyBadge`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `UrgencyBadge({ urgency, className })`

`frontend/src/components/Common/UserAvatar.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `UserAvatar`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `UserAvatar({ fullName, username, avatarType = "preset", avatarPreset = "campus", avatarImage = "", size = 34, className = "", })`
  - `initialFor(fullName, username)`
  - `seedFor(fullName, username, presetId)`

`frontend/src/components/Dashboard/DashboardPrimitives.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `DashboardHero`
  - `DashboardStatGrid`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `DashboardHero({ id = "dashboard", tone = "campus", className = "", children })`
  - `DashboardStatCard({ item })`
  - `DashboardStatGrid({ items, className = "" })`
  - `StatCardDetail({ item })`
  - `resolveTrend(trend)`
  - `toMotionId(item, index)`

`frontend/src/components/Dashboard/DashboardShell.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `DashboardShell`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `DashboardShell({ children })`
  - `getSections()`
  - `handlePreferenceSync(event)`
  - `handleSectionChange(sectionId, label)`
  - `handleViewportChange(event)`
  - `scheduleUpdate()`
  - `toggleCollapse()`
  - `updateActiveSection()`

`frontend/src/components/Dashboard/MotionCardSurface.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `MotionCardSurface`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `MotionCardSurface({ cardId, sectionId, as = "article", className = "", contentClassName = "", children, trackSection = false, morphOnClick = false, detailTitle, detailContent, modalWidth = "max-w-3xl", panelClassName = "", })`
  - `handleKeyDown(event)`
  - `handleOpen(event)`
  - `renderDetailContent(detailContent, close)`
  - `shouldIgnoreCardOpen(event)`

`frontend/src/components/Dashboard/NotificationDropdown.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `NotificationDropdown`
* Classes:
  - No class declarations detected.

* Functions/Components:
  - `NotificationDropdown({ notifications = [], unreadCount = 0, loading = false, error = "", onClose, onOpenNotification, onMarkAllRead, })`
  - `configFor(type)`

`frontend/src/components/Dashboard/ProfileSettingsModal.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `ProfileSettingsModal`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ProfileSettingsModal({ open, onClose, initialTab = "profile", auth, profilePreferences, onSaveProfile, theme, toggleTheme, })`
  - `applyDashboardPreferences()`
  - `handleAvatarUpload(event)`
  - `handleSaveProfile()`
  - `readReduceMotionPreference()`
  - `readSidebarCollapsedPreference()`

`frontend/src/components/Dashboard/Sidebar.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `Sidebar`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `NavItem({ item, collapsed, active, onSelect })`
  - `Sidebar({ isOpen, onClose, collapsed, onToggleCollapse, activeSection, onSectionChange })`
  - `handleLogout()`
  - `handleTouchEnd(event)`
  - `handleTouchStart(event)`
  - `sectionForRole(role)`

`frontend/src/components/Dashboard/TopBar.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `TopBar`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `TopBar({ onMenuClick, isMenuOpen = false, activeSectionLabel, frameClassName = "px-4 sm:px-6 lg:px-8 xl:px-10", })`
  - `handleClickOutside(event)`
  - `handleDashboardNavigate(event)`
  - `handleLogout()`
  - `openNotification(notification)`
  - `openNotificationLink(rawUrl)`
  - `openProfileModal(tab = "profile")`
  - `readReduceMotionPreference()`
  - `saveProfile({ fullName, avatarType, avatarPreset, avatarImage })`

`frontend/src/components/Dashboard/scrollToDashboardSection.js`

* Language: `js`
* Purpose: Reusable UI component
* Exports:
  - `scrollToDashboardSection`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `getDashboardTopOffset()`
  - `scrollToDashboardSection(sectionId, behavior = "smooth")`

`frontend/src/components/Landing/CampusFixLogo.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `CampusFixLogo`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `CampusFixLogo({ size = "md" })`

`frontend/src/components/Landing/Footer.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `Footer`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `DiscordIcon({ className = "" })`
  - `Footer()`
  - `InstagramIcon({ className = "" })`
  - `LinkedInIcon({ className = "" })`
  - `WhatsAppIcon({ className = "" })`
  - `XBrandIcon({ className = "" })`
  - `YouTubeIcon({ className = "" })`

`frontend/src/components/Landing/Navbar.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `Navbar`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `Navbar({ links = [] })`
  - `getNavTopOffset()`
  - `goToSection(href)`
  - `onScroll()`

`frontend/src/components/Landing/useScrollReveal.js`
* Language: `js`
* Purpose: Reusable UI component
* Exports:
  - `useScrollReveal`
* Classes:
  - No class declarations detected.

* Functions/Components:
  - `useScrollReveal(threshold = 0.15)`

`frontend/src/components/hooks/use-image-upload.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `useImageUpload`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `useImageUpload({ onUpload }: UseImageUploadProps = {})`

`frontend/src/components/tickets/TicketTimeline.jsx`
* Language: `jsx`
* Purpose: Reusable UI component
* Exports:
  - `TicketTimeline`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `TicketTimeline({ logs = [] })`

`frontend/src/components/ui/button.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `Button`
  - `buttonVariants`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/components/ui/dock-two.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `Dock`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/components/ui/expandable-tabs.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `ExpandableTabs`
* Classes:
  - No class declarations detected.
* Functions/Components:
    - `ExpandableTabs({ tabs, className, activeColor = "text-campus-700 dark:text-campus-300", onChange, defaultIndex =
null, }: ExpandableTabsProps)`
  - `SeparatorEl()`
  - `handleSelect(index: number)`

`frontend/src/components/ui/fullscreen-calendar.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `FullScreenCalendar`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `FullScreenCalendar({ data, onCreateEvent, }: FullScreenCalendarProps)`
  - `goToToday()`
  - `nextMonth()`
  - `previousMonth()`

`frontend/src/components/ui/hover-gradient-nav-bar.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `default function`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/components/ui/input.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `Input`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/components/ui/separator.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `Separator`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/components/ui/theme-toggle.tsx`
* Language: `tsx`
* Purpose: Reusable UI component
* Exports:
  - `ThemeToggle`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ThemeToggle({ className, isDark: isDarkProp, onToggle }: ThemeToggleProps)`
  - `setNext(nextDark: boolean)`

`frontend/src/context/AuthContext.jsx`
* Language: `jsx`

* Purpose: Project source code
* Exports:
  - `AuthProvider`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AuthProvider({ children })`
  - `bootstrap()`
  - `isMfaChallenge(data)`
  - `normalizeSession(data)`
  - `parseError(error, fallback = "Request failed. Please try again.")`
  - `roleHome(role)`

`frontend/src/context/ThemeContext.jsx`
* Language: `jsx`
* Purpose: Project source code
* Exports:
  - `ThemeProvider`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ThemeProvider({ children })`

`frontend/src/context/auth-context.js`
* Language: `js`
* Purpose: Project source code
* Exports:
  - `AuthContext`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/context/theme-context.js`
* Language: `js`
* Purpose: Project source code
* Exports:
  - `ThemeContext`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/hooks/use-media-query.js`
* Language: `js`
* Purpose: Frontend hook/state logic
* Exports:
  - `useMediaQuery`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `getSnapshot()`
  - `subscribe(onStoreChange)`
  - `useMediaQuery(query)`

`frontend/src/hooks/useAuth.js`
* Language: `js`

* Purpose: Frontend hook/state logic
* Exports:
  - `useAuth`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `useAuth()`

`frontend/src/hooks/useNotifications.js`
* Language: `js`
* Purpose: Frontend hook/state logic
* Exports:
  - `useNotifications`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `syncVisibility()`
  - `useNotifications(enabled = true)`

`frontend/src/hooks/useReducedMotionPreference.js`
* Language: `js`
* Purpose: Frontend hook/state logic
* Exports:
  - `useReducedMotionPreference`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `hasReduceMotionClass()`
  - `updatePreference()`
  - `useReducedMotionPreference()`

`frontend/src/hooks/useTheme.js`
* Language: `js`
* Purpose: Frontend hook/state logic
* Exports:
  - `useTheme`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `useTheme()`

`frontend/src/hooks/useTickets.js`
* Language: `js`
* Purpose: Frontend hook/state logic
* Exports:
  - `useTickets`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `syncVisibility()`
  - `useTickets(loader, deps = [], options = {})`

`frontend/src/lib/utils.ts`
* Language: `ts`
* Purpose: Project source code
* Exports:
  - `cn`

* Classes:
  - No class declarations detected.
* Functions/Components:
  - `cn(...inputs: Array<string | undefined | null | false>)`

`frontend/src/main.jsx`
* Language: `jsx`
* Purpose: Project source code
* Exports:
  - No named/default exports detected.
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/pages/AboutPage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `AboutPage`
  - `default AboutPage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AboutPage()`
  - `applyHashOffset()`
  - `resolveNavOffset()`

`frontend/src/pages/AcceptInvitePage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `AcceptInvitePage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AcceptInvitePage()`
  - `fetchSuggestions()`
  - `onSubmit(values)`

`frontend/src/pages/AdminDashboard.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `AdminDashboard`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `AdminDashboard()`
  - `approveTicket()`
  - `askConfirm(title, message, onConfirm)`
  - `assignTicket()`
  - `buildTicketSearchText(ticket)`
  - `formatScopeValue(value)`
  - `getSlaStatus(ticket)`
  - `handleBroadcast(form)`

  - `handleCancelScheduled(id)`
  - `handleGlobalExport(format)`
  - `handleInviteStaff(form)`
  - `handleSchedule(form)`
  - `inRange(v, s, e)`
  - `openTicket(ticketId)`
  - `overrideStatus()`
  - `refreshScheduledEvents()`
  - `refreshUsers()`
  - `rejectTicket()`
  - `runAction(task)`
  - `trendPercent(current, previous)`

`frontend/src/pages/ContactSupportPage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `ContactSupportPage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ContactSupportPage()`
  - `fieldClass(hasError)`
  - `onSubmit(values)`

`frontend/src/pages/ForgotPasswordPage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `ForgotPasswordPage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ForgotPasswordPage()`
  - `fieldClass(hasError)`
  - `onSubmit(values)`

`frontend/src/pages/LandingPage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `LandingPage`
  - `default LandingPage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `LandingPage()`

`frontend/src/pages/LoginPage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `LoginPage`
* Classes:
  - No class declarations detected.
* Functions/Components:

- `LoginPage()`
- `destination(role)`
- `fieldClass(hasError)`
- `onSubmit(values)`
- `resendMfa()`
- `resetMfaFlow()`
- `verifyMfa()`

`frontend/src/pages/MaintenanceDashboard.jsx`

* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `MaintenanceDashboard`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `MaintenanceDashboard()`
  - `OperationalBriefDetail({ queueHealth, avgRating, avgRatingLoading, averageResolutionHours, resolvedToday, activeCount, resolvedCount })`
    - `WorkQueueCard({ ticket, note, afterPhoto, actionState, onNoteChange, onPhotoChange, onOpenTicket, onUpdateStatus, onRespondAssignment })`
  - `buildTicketSearchText(ticket)`
  - `formatRemaining(hours)`
  - `getSlaRemaining(ticket)`
  - `loadAvgRating()`
  - `openTicket(ticketId)`
  - `respondToAssignment(ticket, accepted)`
  - `updateStatus(ticket, status)`

`frontend/src/pages/NotFoundPage.jsx`

* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `NotFoundPage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `NotFoundPage()`

`frontend/src/pages/PrivacyPage.jsx`

* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `PrivacyPage`
  - `default PrivacyPage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `PrivacyPage()`

`frontend/src/pages/RegisterPage.jsx`

* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `RegisterPage`

* Classes:
  - No class declarations detected.
* Functions/Components:
  - `RegisterPage()`
  - `fetchSuggestions()`
  - `fieldClass(hasError)`
  - `normalizeRegisterSubmitError(message)`
  - `onSubmit(values)`

`frontend/src/pages/ResetPasswordPage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `ResetPasswordPage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `ResetPasswordPage()`
  - `onSubmit(values)`

`frontend/src/pages/StudentDashboard.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `StudentDashboard`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `RecentActivityDetail({ tickets })`
  - `StudentDashboard()`
  - `TicketTracker({ ticket })`
  - `TrackerDetail({ ticket, statusCounts, openUrgentCount, averageResolutionHours })`
  - `createDefaultForm({ serviceDomainKey = "", requestTypeId = "", buildingId = "", } = {})`
  - `formatAverageDuration(hours)`
  - `handleOpenComposer()`
  - `openTicket(ticketId)`
  - `submitRating(event)`
  - `submitTicket(event)`

`frontend/src/pages/TermsPage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `TermsPage`
  - `default TermsPage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `TermsPage()`

`frontend/src/pages/VerifyEmailPage.jsx`
* Language: `jsx`
* Purpose: Frontend route/page component
* Exports:
  - `VerifyEmailPage`
* Classes:

  - No class declarations detected.
* Functions/Components:
  - `VerifyEmailPage()`
  - `fieldClass(hasError)`
  - `resendCode()`
  - `verifyCode(values)`

`frontend/src/queries/catalogQueries.js`
* Language: `js`
* Purpose: Project source code
* Exports:
  - `catalogKeys`
  - `useActiveBuildingsQuery`
  - `useActiveRequestTypesQuery`
  - `useAdminBuildingsQuery`
  - `useAllRequestTypesQuery`
  - `useAllSupportCategoriesQuery`
  - `useOperationalBuildingsQuery`
  - `useServiceDomainsQuery`
  - `useSupportCategoriesQuery`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `useActiveBuildingsQuery()`
  - `useActiveRequestTypesQuery(serviceDomainKey)`
  - `useAdminBuildingsQuery()`
  - `useAllRequestTypesQuery()`
  - `useAllSupportCategoriesQuery()`
  - `useOperationalBuildingsQuery()`
  - `useServiceDomainsQuery()`
  - `useSupportCategoriesQuery()`

`frontend/src/services/adminConfigService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `adminConfigService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/services/analyticsService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `analyticsService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/services/apiClient.js`
* Language: `js`
* Purpose: Frontend API/client service
* Exports:

  - `apiBaseUrl`
  - `configureApiClientAuth`
  - `default apiClient`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `configureApiClientAuth({ accessTokenGetter, refreshSession, clearSession })`
  - `getAccessToken()`
  - `isAuthPage()`
  - `isAuthRefreshEndpoint(url = "")`
  - `isPublicEndpoint(url = "")`

`frontend/src/services/authService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `authService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/services/buildingService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `buildingService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/services/catalogService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `catalogService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/services/exportService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `exportToCSV`
  - `exportToPDF`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `downloadBlob(content, filename, mimeType)`
  - `exportToCSV(data, columns, filename = "export")`
  - `exportToPDF(data, columns, filename = "export", title = "Report")`

`frontend/src/services/notificationService.js`
* Language: `js`

* Purpose: Business/service logic
* Exports:
  - `notificationService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/services/supportService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `supportService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/services/ticketService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `ticketService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `buildQuery(params = {})`

`frontend/src/services/userService.js`
* Language: `js`
* Purpose: Business/service logic
* Exports:
  - `userService`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/utils/constants.js`
* Language: `js`
* Purpose: Shared utility/helper logic
* Exports:
  - `CATEGORIES`
  - `ROLES`
  - `STATUSES`
  - `STATUS_COLORS`
  - `URGENCY_COLORS`
  - `URGENCY_LEVELS`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/utils/helpers.js`
* Language: `js`
* Purpose: Shared utility/helper logic
* Exports:

  - `formatDate`
  - `titleCase`
  - `toHours`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `formatDate(value)`
  - `titleCase(value)`
  - `toHours(start, end)`

`frontend/src/utils/passwordPolicy.js`
* Language: `js`
* Purpose: Shared utility/helper logic
* Exports:
  - `evaluatePassword`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `evaluatePassword(password, { username = "", email = "", fullName = "" } = {})`
  - `hasDigit(value)`
  - `hasLower(value)`
  - `hasSymbol(value)`
  - `hasUpper(value)`
  - `hasWhitespace(value)`
  - `push(value)`
  - `tokenSet(username, email, fullName)`

`frontend/src/utils/profilePreferences.js`
* Language: `js`
* Purpose: Shared utility/helper logic
* Exports:
  - `AVATAR_PRESETS`
  - `loadProfilePreferences`
  - `normalizeAvatarPreset`
  - `resolveAvatarPreset`
  - `saveProfilePreferences`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `hasPreset(presetId)`
  - `keyFor(username)`
  - `loadProfilePreferences(username)`
  - `normalizeAvatarPreset(presetId)`
  - `resolveAvatarPreset(presetId)`
  - `saveProfilePreferences(username, prefs)`

`frontend/src/utils/storage.js`
* Language: `js`
* Purpose: Shared utility/helper logic
* Exports:
  - `themeStorage`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/src/utils/ticketPresentation.js`

* Language: `js`
* Purpose: Shared utility/helper logic
* Exports:
  - `getTicketBuildingCode`
  - `getTicketBuildingName`
  - `getTicketLocationSummary`
  - `getTicketRequestTypeLabel`
  - `getTicketServiceDomainKey`
  - `getTicketServiceDomainLabel`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `getTicketBuildingCode(ticket)`
  - `getTicketBuildingName(ticket)`
  - `getTicketLocationSummary(ticket)`
  - `getTicketRequestTypeLabel(ticket)`
  - `getTicketServiceDomainKey(ticket)`
  - `getTicketServiceDomainLabel(ticket)`

`frontend/tailwind.config.js`

* Language: `js`
* Purpose: Project source code
* Exports:
  - No named/default exports detected.
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `addVariablesForColors({ addBase, theme })`
  - `flattenColorPalette(colors, prefix = "")`

`frontend/tests/e2e/dashboard-flows.spec.js`

* Language: `js`
* Purpose: Project source code
* Exports:
  - No named/default exports detected.
* Classes:
  - No class declarations detected.
* Functions/Components:
  - `building(id, name, code, active = true)`
  - `json(route, body, status = 200)`
  - `makeSession(role, overrides = {})`
  - `makeTicket({ id, title, serviceDomainKey = "IT", requestTypeLabel = "Network Support", requestTypeId = 11, buildingId = 1, buildingName = "Main Library", buildingCode = "LIB", location = "Floor 2", urgency = "MEDIUM", status = "SUBMITTED", createdBy, assignedTo = null, createdAt = NOW, updatedAt = NOW, resolvedAt = null, imageUrl = null, afterImageUrl = null, description = "Campus maintenance issue.", })`
  - `mockCommonApi(page, session, extraHandler)`
  - `requestBody(request)`
  - `requestType(id, label, serviceDomainKey)`
  - `ticketUser(id, username, fullName, role)`

`frontend/tests/e2e/public-auth.spec.js`

* Language: `js`
* Purpose: Project source code
* Exports:
  - No named/default exports detected.

* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.

`frontend/vite.config.js`

* Language: `js`
* Purpose: Project source code
* Exports:
  - `default defineConfig`
* Classes:
  - No class declarations detected.
* Functions/Components:
  - No function declarations detected.